Software Design Specification

AI Powered Hospital Information Management System

Project Code:

HIMS-AI-2526

Internal Advisor:

Mr. Aamir Zia

Project Manager:

Dr. Muhammad Ilyas

Project Team:

Ali Akbar	BSCS51F22R036	(TL)
Muhammad Najee Ullah Noon	BSCS51F22S043	

Submission Date:

Dec 15, 2025

Project Manager's Signature

Document Information

Category	Information
Customer	Computer Science , University Of Sargodha
Project	<Hospital Information Management System >
Document	Requirement Specifications
Document Version	1.0
Identifier	HIMS-AI-2526
Status	Draft
Author(s)	Ali Akbar Muhammad Najee Ullah Noon
Approver(s)	PM
Issue Date	Dec. 15, 2025
Document Location	
Distribution	1. Advisor 2. Project Manager 3. Project Office

Definition of Terms, Acronyms and Abbreviations

Term	Description
HIMS	Hospital Information Management System
RS	Requirements Specifications
HIPPA	Health Insurance Portability and Accountability Act
GDPR	General Data Protection Regulation
LAN	Local Area Network
CNIC	Computerized National Identity Card

Table of Contents

1. Introduction

1.1 Purpose of Document

The document presents the design requirements of the project "AI Powered Hospital Information Management System". It gives a detailed explanation of system's functionality, its design and how the AI-based solution will automate the patient data management and streamline hospital workflows. Audience covers the project stakeholders, developers, healthcare IT managers and hospital administrators. It serves as reference to ensure that all parties have a unified understanding of the system's objectives, architecture, constraints and operational behaviors. The document also ensures that the design aligns with HIPPA and GDPR. It will act as a guide during development, testing and deployment phases ensuring that the final product fulfills its intended purpose of reducing administrative burden and improving hospital efficiency.

1.2 Project Overview

Modern hospitals often struggle with inefficiencies caused by manual data entry, fragmented record-keeping, and time-consuming administrative processes. These challenges lead to delays in patient care, errors in documentation, and an increased workload on medical staff.

To address these issues, the AI-Powered Hospital Information Management System (HIMS) is proposed as an intelligent, automated platform that integrates speech recognition and natural language processing to streamline hospital workflows.

The system functions as follows:

1. **AI-Based Conversation Recording and Analysis:**
   The system listens to doctor–patient interactions in real time, identifies the speaker, and extracts key medical information such as symptoms, diagnosis, prescribed treatments, and follow-up details.

2. **Doctor Confirmation and Data Storage:**
   Before saving, the extracted data is displayed to the doctor for verification. Once confirmed, it is automatically stored in a centralized hospital database.

3. **Dedicated Dashboards:**

- **Doctor Dashboard:** Enables doctors to view patient histories, review notes, and manage consultations.

- **Admin Dashboard:** Allows hospital administrators to manage records, monitor data flow, and ensure system compliance.

- **Patient Portal:** Provides patients with access to their medical history, prescriptions, and reports.

4. **Appointment Management**: Real-time queue tracking and appointment management help minimize overcrowding and waiting time in hospitals.

5. **Medical History**: Digital record management ensures easy and secure access to patient history, prescriptions, and billing information.

The core objective of this project is to reduce administrative workload, improve record accuracy, and enable real-time access to patient information. By leveraging advanced technologies such as Deepgram for speech-to-text processing, OpenAI GBT 4.1 for data extraction, and PostgreSQL for secure data management, the proposed system enhances hospital efficiency and promotes the adoption of AI-driven healthcare solutions.

1.3 Scope

The scope of the AI-powered Hospital Information Management System (HIMS) is defined by its focus on improving patient record management, reducing administrative workload, and supporting accurate medical documentation through AI assistance, while keeping doctors in control of validation. The project is specifically designed for healthcare institutions and has clear boundaries on what it will and will not cover.

Inclusions (What the System Will Do):

- **AI-Assisted Medical Note Creation:** "The system captures doctor-patient conversations, highlights relevant medical details (symptoms, diagnoses, tests), and presents them for doctor confirmation before saving to the database [4]."
- **Smart Appointment and Queue Management:** Patients can book appointments on a first-come-first-serve basis and monitor their queue position in real-time to minimize hospital waiting time and overcrowding.
- **Controlled Patient Self-Registration:** Patients can create their own accounts through the registration system; however, the accounts will remain inactive until verified and approved by authorized hospital staff to ensure security and authenticity of patient information.
- **Centralized Patient Record Management:** Hospitals can maintain structured medical histories, accessible to authorized staff across visits.
- **Prescription Management System:** The platform stores doctor-specific prescription history, allowing healthcare providers to review previous medications and treatment records for continuity of care.
- **Integrated Staff Dashboard:** Enables staff to verify or edit AI-suggested notes, manage patient data, and oversee hospital operations such as scheduling and reporting.
- **Integrated Communication Module:** The system provides a secure messaging platform for healthcare-related communication, follow-ups, notifications, and information exchange within the hospital management environment.
- **Secure Patient Access:** Patients can receive their reports digitally through their profile or via staff-provided printed documents.

- **Transparent Billing and Report Access:** Patients can access billing information, payment details, and downloadable medical reports digitally through their accounts.
- **Healthcare Data Analysis:** Aggregated hospital data can be used to analyze patient trends, workload distribution, consultation frequency, and operational efficiency.

Exclusions (What the System Will Not Do):

- **Autonomous Medical Decision-Making:** The system will not independently diagnose diseases, prescribe medicines, or recommend treatments without doctor involvement and approval.
- **Cross-Hospital Data Sharing:** Patient data will not automatically transfer between hospitals unless formally integrated with external systems.
- **Non-Medical Hospital Operations:** The system will not handle unrelated tasks such as payroll, pharmacy stock management, or facility maintenance.
- **Emergency Response Management:** The platform is not intended to function as an emergency dispatch or ambulance coordination system.
- **Advanced AI Disease Prediction:** The current version of the system will not include predictive analytics for disease forecasting or automated medical risk assessment.
- **Multilingual Limitation:** The system currently supports English & Urdu only and does not provide support for additional regional or international languages in the present implementation.

2. Design Considerations

2.1 Assumptions and Dependencies

Assumptions

- The hospital's medical staff and administrative personnel will cooperate fully in adopting the new HIMS platform and provide accurate input data during initial setup and daily usage.
- The hospital's existing hardware infrastructure (computers, microphones, and secure servers) will meet the minimum system requirements for smooth operation.
- Internet connectivity within the hospital premises will remain stable to enable real-time data synchronization and AI model processing.
- Doctor-patient conversation recordings will be conducted in environments with minimal background noise to ensure the accuracy of the speech-to-text and entity extraction modules.

- All users—including doctors, nurses, and administrative staff—will receive basic training on how to use the system effectively.
- The hospital's policies and workflows will remain consistent during the system's development and deployment phases, ensuring alignment between technical and operational procedures.

Dependencies

- The AI modules depend on external speech-to-text and natural language processing APIs for transcribing and extracting key medical data from doctor-patient conversations.
- The system relies on a stable connection to the hospital's central database for data storage and retrieval of patient records, diagnoses, and treatment histories.
- Regular data updates and maintenance must be performed by authorized IT personnel to ensure accuracy and prevent inconsistencies.
- Integration with existing hospital modules (e.g., billing, pharmacy, and lab systems) depends on the availability and compatibility of their APIs or export functionalities.
- The system's deployment and maintenance depend on compliance with healthcare regulations such as HIPAA or local data protection laws to ensure security and confidentiality.
- Timely software updates, API key renewals, and support from external AI service providers are necessary to maintain full functionality and performance.

2.2 Risks and Volatile Areas

1. Integration with External Systems

Risk: The system may require integration with external services such as speech recognition (ASR), NLP engines, email services, or future healthcare APIs. Changes in external APIs or protocols may affect system functionality.

Design Response:

- Use modular API integration architecture.
- Define clear API communication interfaces.
- Separate external service logic from core system modules to simplify future updates and maintenance.

2. Technology and Infrastructure Changes

Risk: Future upgrades in databases, hosting environments, or software frameworks may impact system compatibility and performance.

Design Response:

- Use a layered system architecture for easier maintenance and upgrades.
- Apply database abstraction through Django ORM.
- Maintain modular backend and frontend components to support future scalability.

3. Performance and Scalability Demands

Risk: Increasing patient records, concurrent users, and AI-based voice processing may reduce system performance.

Design Response:

- Optimize database queries and API responses.
- Use efficient data caching where necessary.
- Process voice transcription and report generation efficiently to reduce server load.

4. Security Threats

Risk: Unauthorized access, data breaches, or misuse of sensitive patient information may compromise system security.

Design Response:

- Implement role-based access control for different user types.
- Encrypt sensitive data during transmission and storage.
- Use secure authentication and session management mechanisms.
- Regularly monitor and update system security measures.

5. Third-Party Service Dependency Risks

Risk: External services such as ASR engines, cloud storage, or email gateways may experience downtime or API changes.

Design Response:

- Design the system with replaceable service integration modules.
- Implement proper error handling and retry mechanisms.
- Maintain backup configurations for critical external services when possible.

6. Operational Risks

Risk: Server failures, internet connectivity issues, or database errors may interrupt hospital operations.

Design Response:

- Maintain regular database backups.
- Implement recovery mechanisms for critical data.
- Monitor server health and system availability.
- Use reliable hosting infrastructure to minimize downtime.

3. System Architecture

3.1 Use Case Diagram

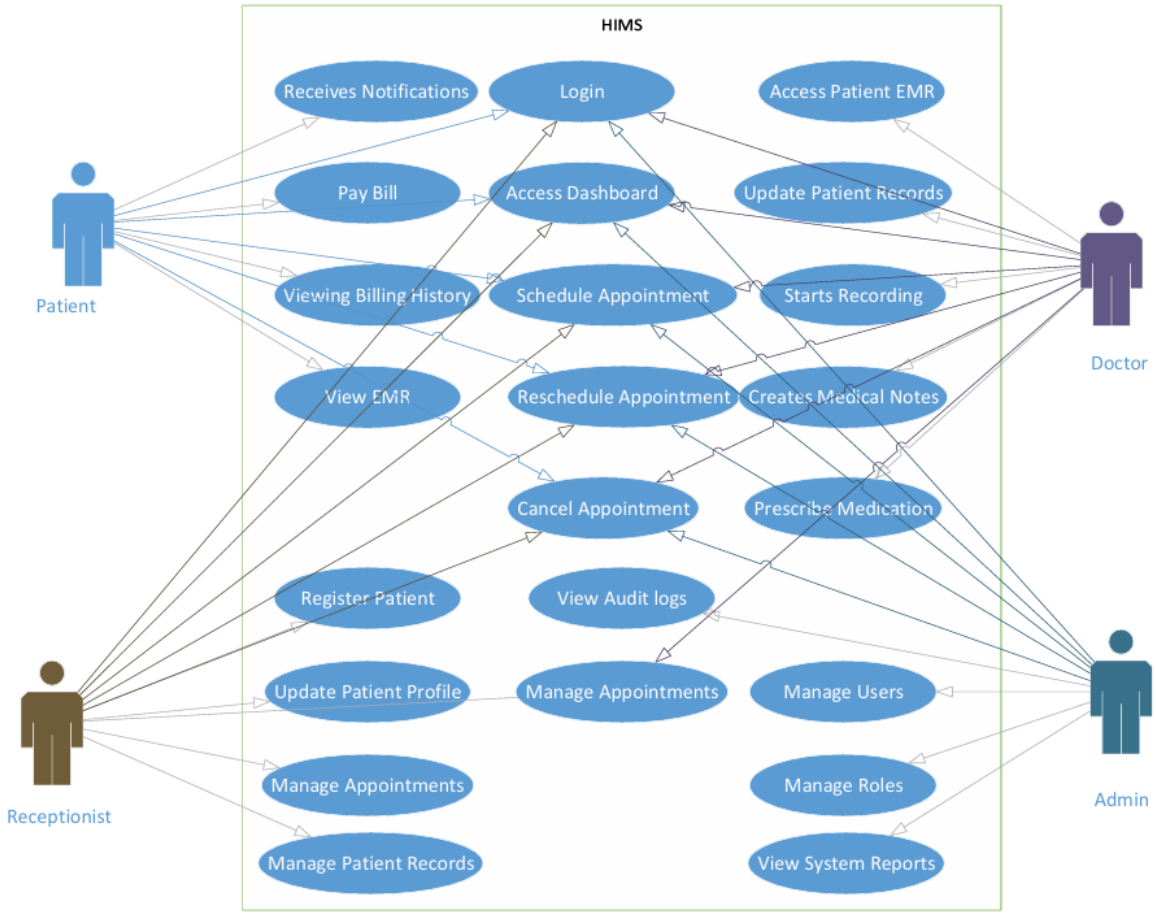

Figure 3.1 Use Case Diagram

3.2 System Level Architecture

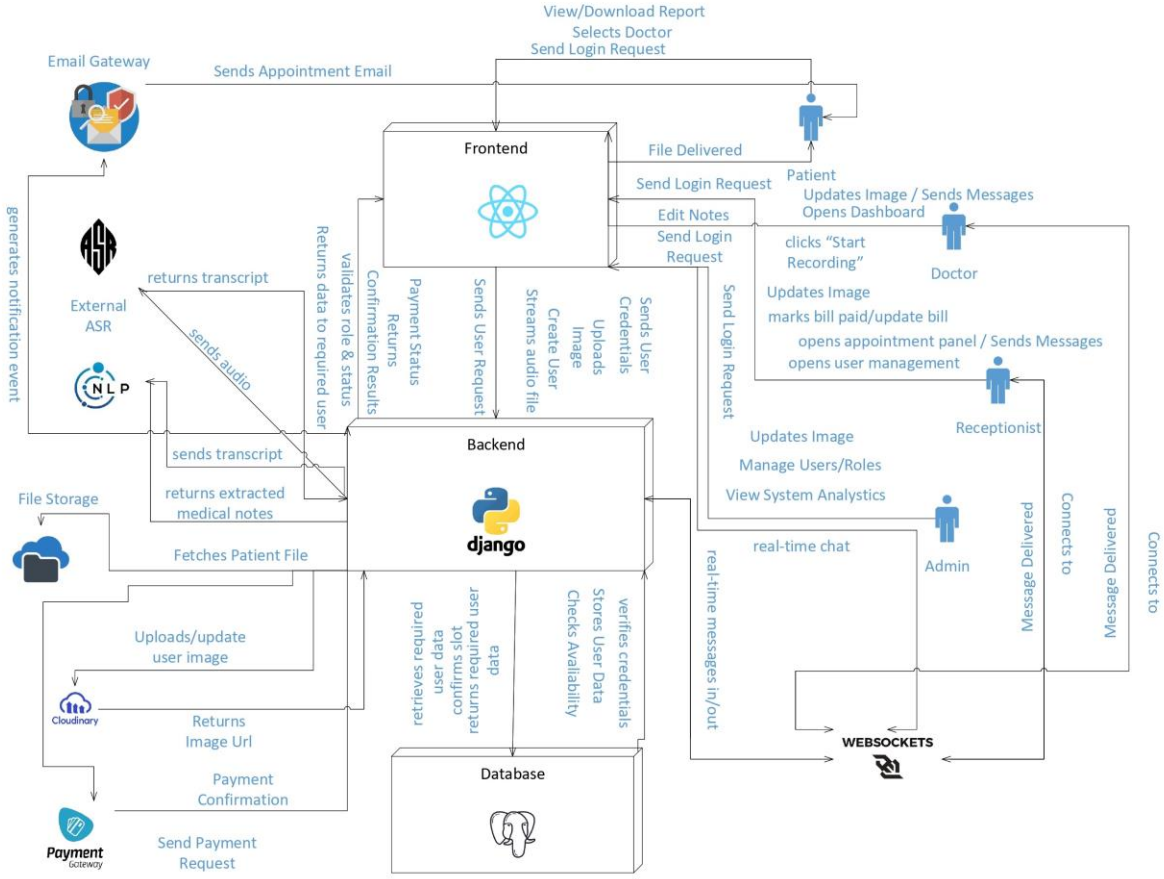

Figure 3.2 System Architecture Diagram

3.3 Sub-System / Component / Module Level Architecture

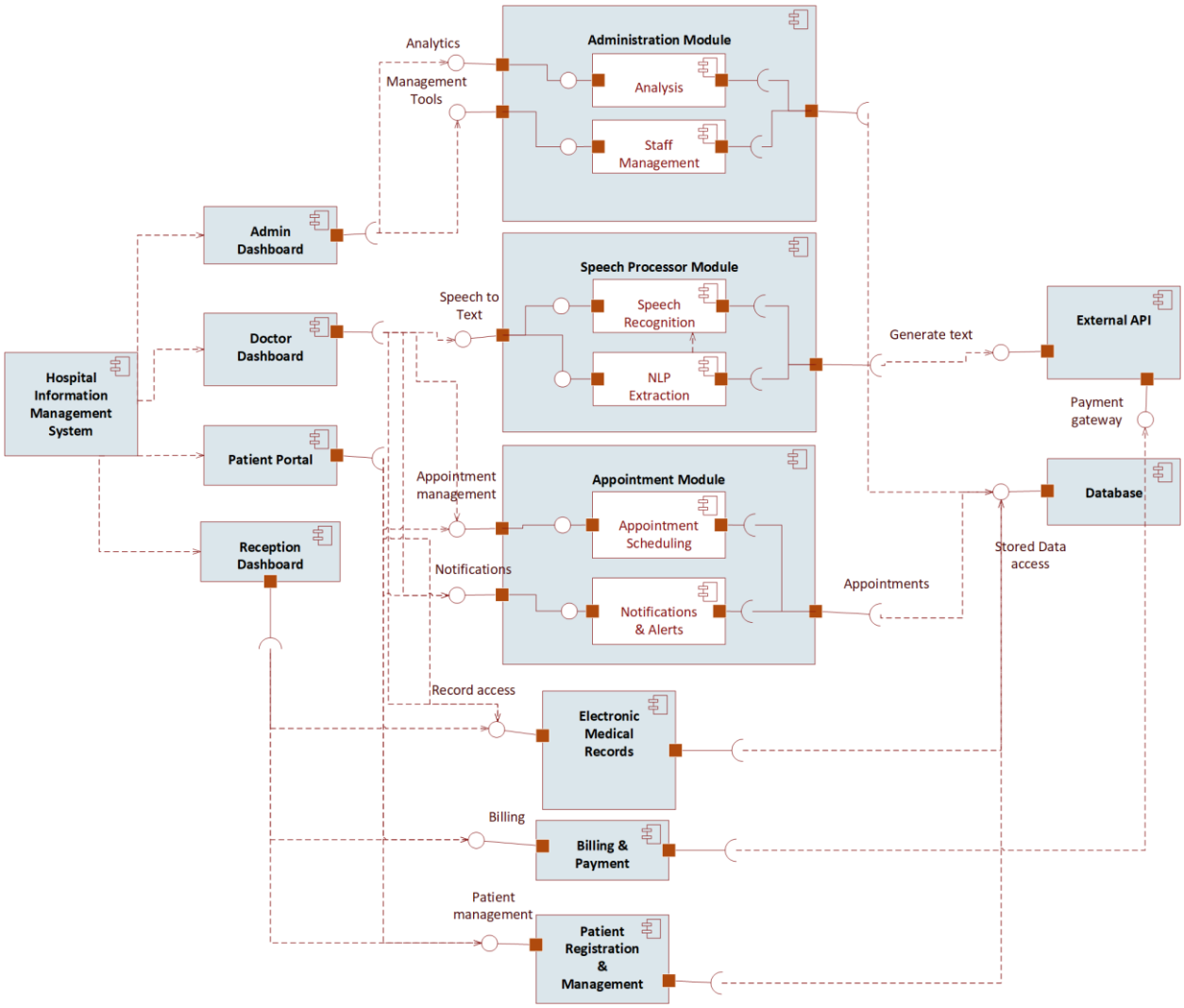

Figure 3.3 Component Diagram

3.4 Sub-Component / Sub-Module Level Architecture (1…n)

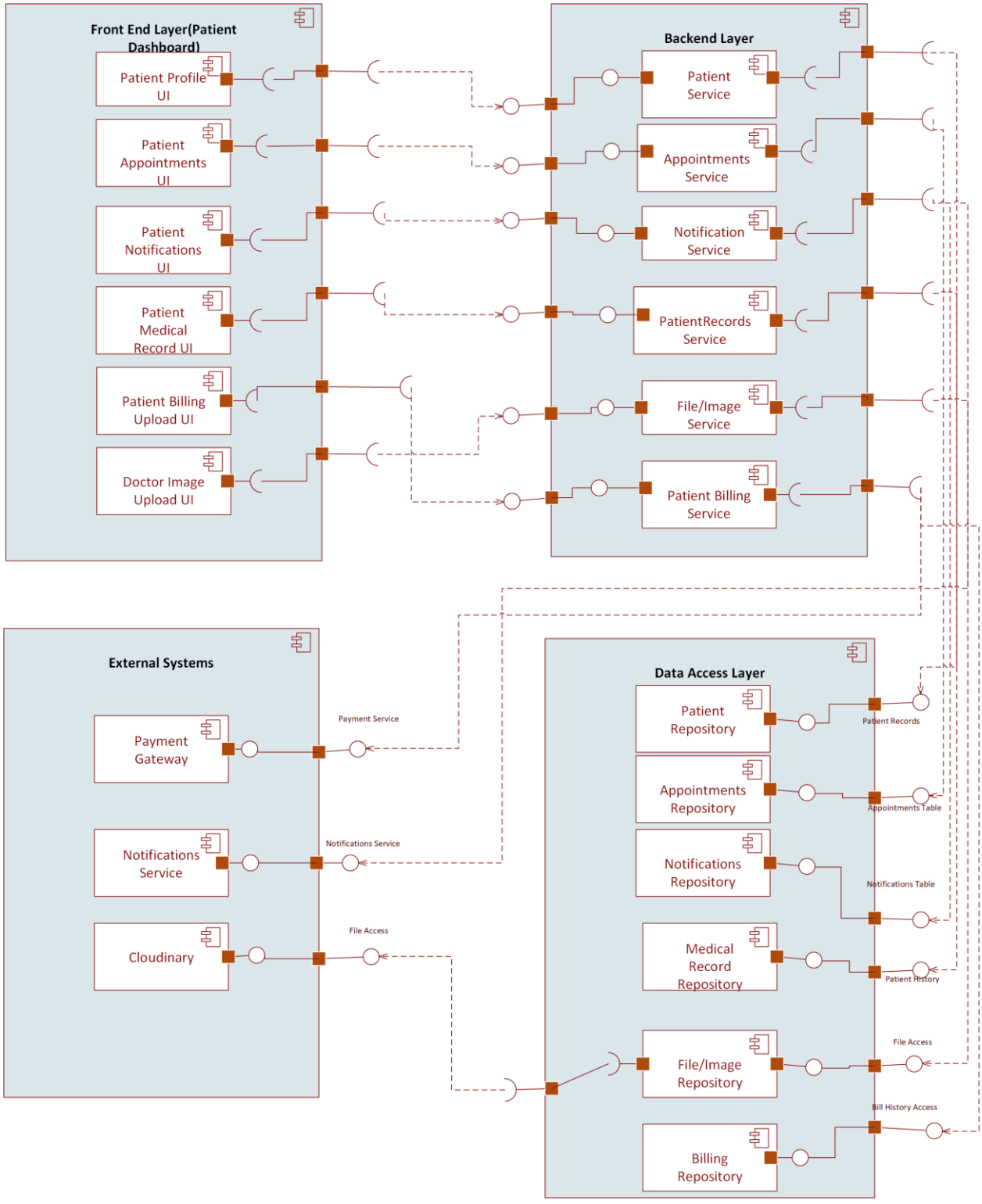

Figure 3.4.1 Patient Component Diagram

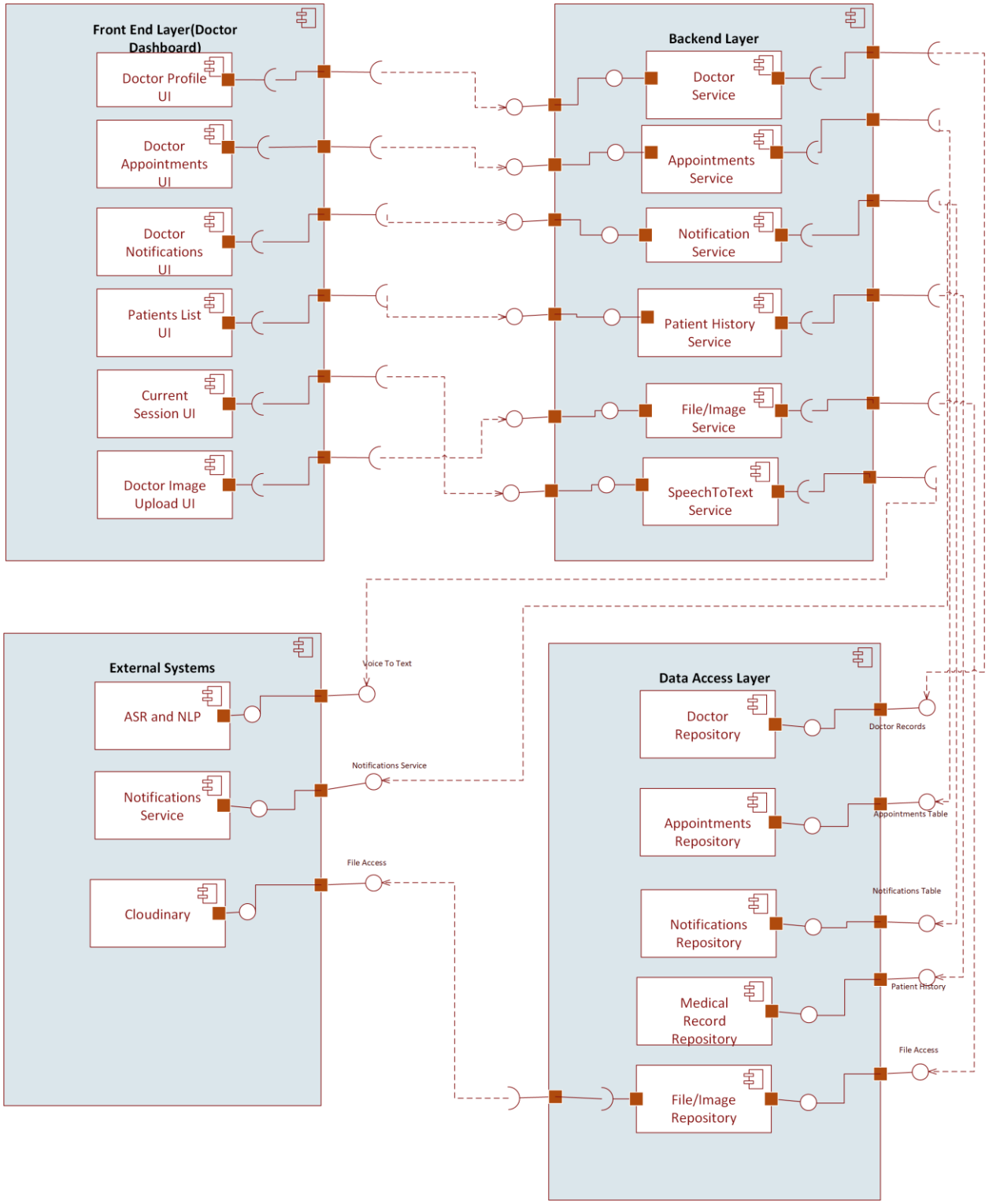

Figure 3.4.2 Doctor Component Diagram

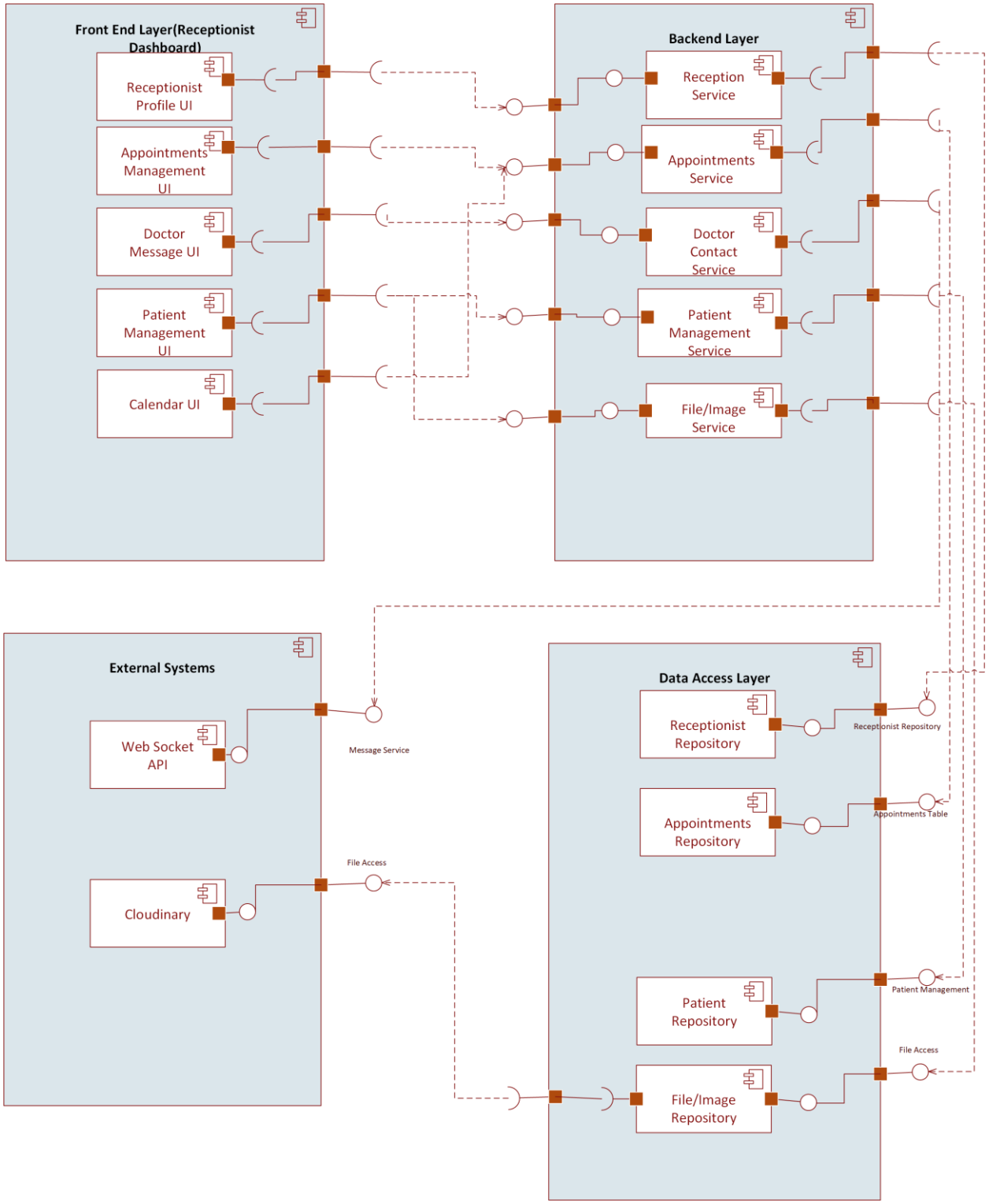

Figure 3.4.3 Receptionist Component Diagram

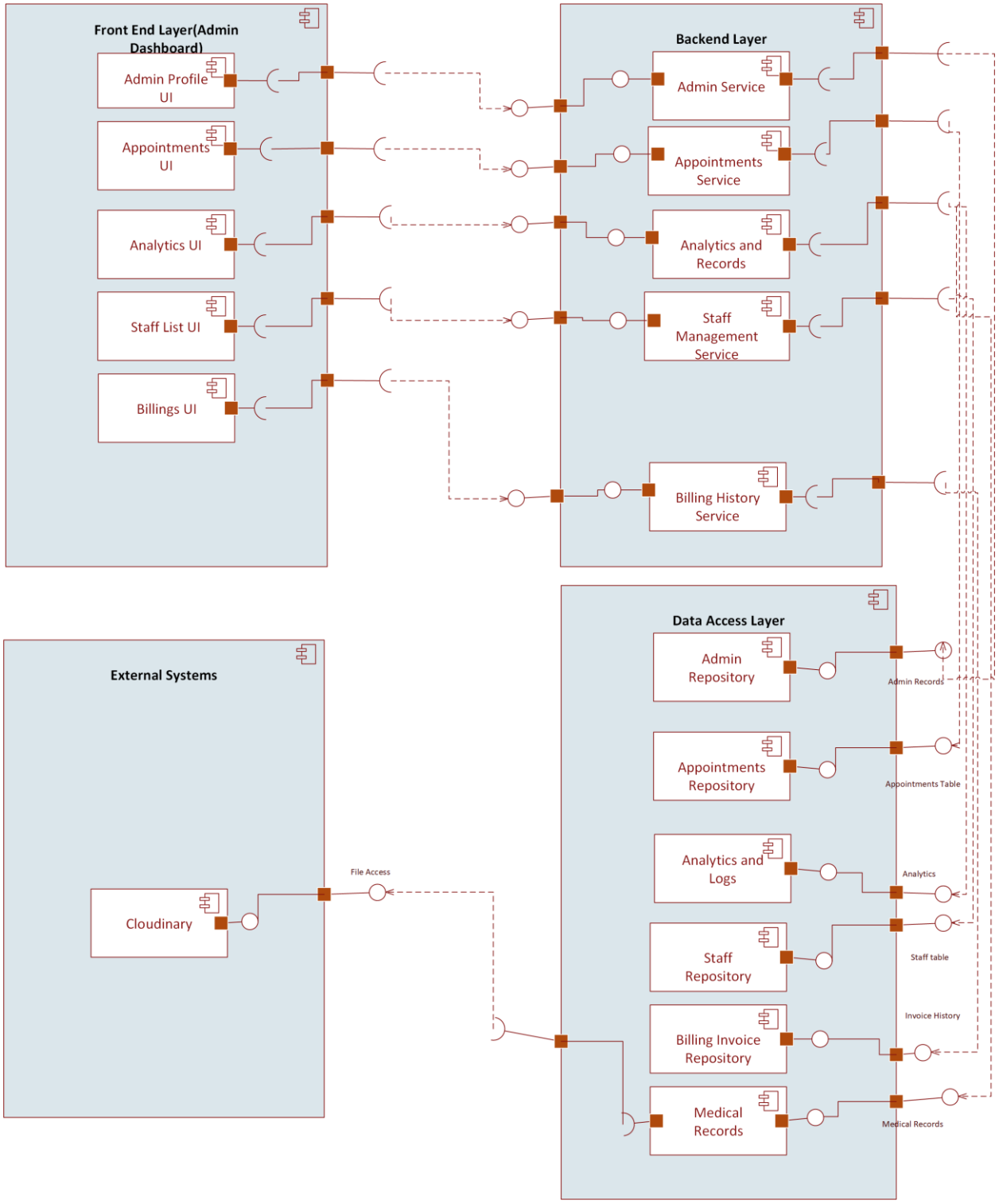

Figure 3.4.4 Admin Component Diagram

4. Design Strategies

4.1 Strategy 1: Modular Layered Architecture

The system is structured into logical layers:

1. **Presentation Layer (Frontend UI)**
   Patient portal, doctor portal, receptionist portal and admin dashboards.
2. **Application Layer (Backend Logic)**
   Appointment handling, patient record management, report processing, ASR pipelines.
3. **Data Access Layer**
   Repository classes, ORM queries, database communication.
4. **Database Layer**
   PostgreSQL for structured hospital data.

Reasoning

- Separates UI, business logic, and database interaction.
- Makes each module easy to modify or extend.
- Reduces complexity and avoids tangled dependencies.

Trade-offs

- Requires discipline to maintain layer boundaries.
- Slight overhead in communication between layers.

If the hospital later introduces **pharmacy**, **lab integration**, or **insurance**, new modules plug into the architecture without affecting existing ones.

4.2 Strategy 2: API-Driven / Service-Oriented Design

All modules communicate using clearly defined **REST APIs**.

Examples:

- /appointments/book
- /patients/update-history

Reasoning

- Clean separation between frontend and backend.
- Easy to integrate with mobile apps, external labs, or third-party hospital devices.

- Allows independent scaling of services later.

Trade-offs

- More API design work initially.
- Network latency between services.

The **voice-to-text ASR service** can run separately without modifying the main backend. This makes the system future-proof for new AI modules.

4.3 Strategy 3: Future System Extension & Plug-in Architecture

The architecture is designed to allow **new workflows and modules** without rewriting existing ones. Strategies used:

- Interface-based design
- Configurable modules
- Separate routing for each domain (patients, appointments, diagnosis, reports)

Reasoning

Hospitals frequently add:

- New lab machines
- New departments
- New data flows (insurance, pharmacy automation)

Trade-offs

- Requires abstraction and planning during early design.
- Some modules might have more configuration overhead.

If we add a **Radiology module**, it simply becomes:

- New routes: /radiology/upload, /radiology/report
- New table: radiology_reports
- No impact on appointments, patients, or ASR pipeline.

4.4 Strategy 4: Unified Data Management Strategy

The system uses a consistent approach to data storage:

- **PostgreSQL** for structured hospital records

- **Cloud storage** for files (images)
- **ACID transactions** for sensitive updates
- **Foreign keys** to prevent inconsistent data
- **Indexed queries** for real-time performance (appointments, patient history)

Reasoning

- Ensures correctness, especially for medical data.
- Enables fast retrieval of patient histories.
- Supports large-scale hospital operations.

Trade-offs

- Requires schema planning.
- Some features (e.g., full-text search) need extra tools.

Appointment booking uses transactions to prevent **double-booking** when multiple receptionists try to schedule the same slot at the same time.

4.5 Strategy 5: Concurrency & Synchronization Controls

Some hospital actions need safe handling of parallel requests:

- Prescription updates
- Audio file uploads → ASR processing
- Simultaneous doctor and receptionist updates on patient record

Concurrency mechanisms used:

- Database locks
- Transaction isolation
- Atomic operations
- Queues for long tasks (ASR, NLP)

Reasoning

- Guarantees consistent data.
- Prevents race conditions.
- Critical for time-sensitive medical operations.

Trade-offs

- High concurrency may reduce performance due to locking.
- Requires careful backend design.

4.6 Strategy 6: Reusability of Components

Common utilities are designed for reuse across modules:

- Logging service
- Authentication middleware
- File upload utility
- Notification/email service
- Validation helpers

Reasoning

- Reduces repeated code.
- Makes maintenance easier.
- Ensures consistent behavior across modules.

Trade-offs

- Shared services must be carefully versioned to avoid breaking modules.

The **file upload service** (used by voice recordings and lab reports) is a single shared component used by multiple modules.

4.7 Strategy 7: Secure-by-Design Architecture

Security is built into the architecture instead of added later:

- Role-based access control (Admin, Doctor, Patient)
- Encrypted communication (HTTPS)
- Input validation
- Audit logging
- Secure authentication tokens

Reasoning

Hospitals handle extremely sensitive patient data.

Trade-offs

- Adds development overhead
- Requires periodic updates and policy checks

Doctors can only access their assigned patient records. Patients cannot view internal notes or doctor comments.

1. Detailed System Design

5.1 Class Diagram

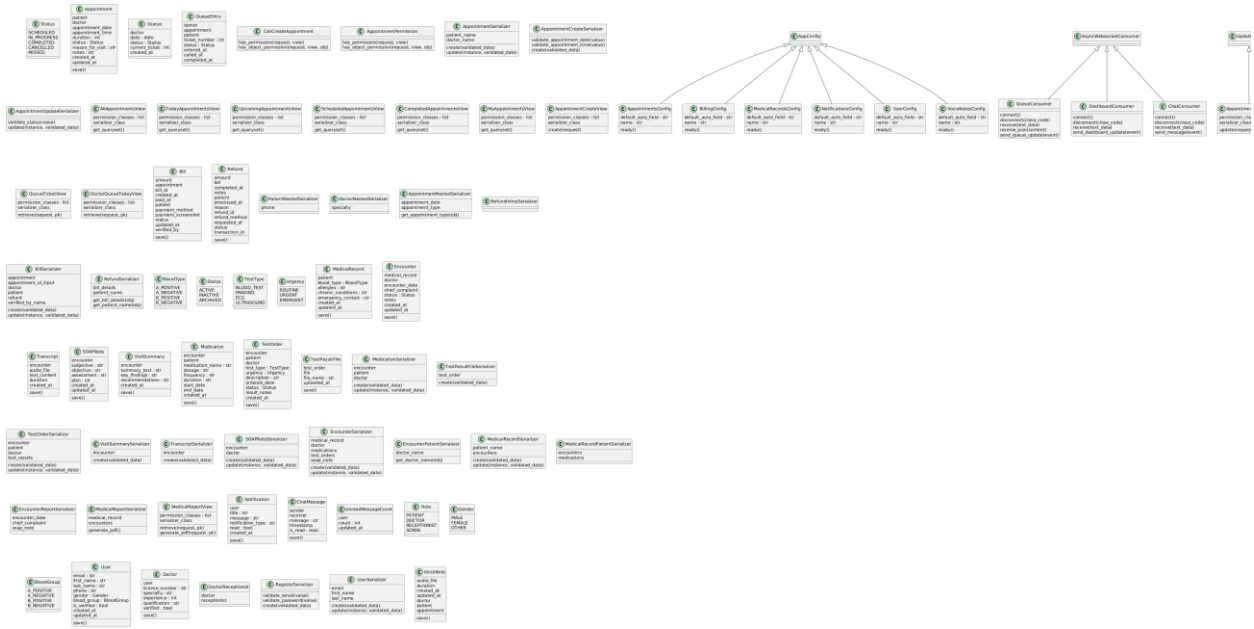

Figure 5.1.1 HIMS Class Diagram

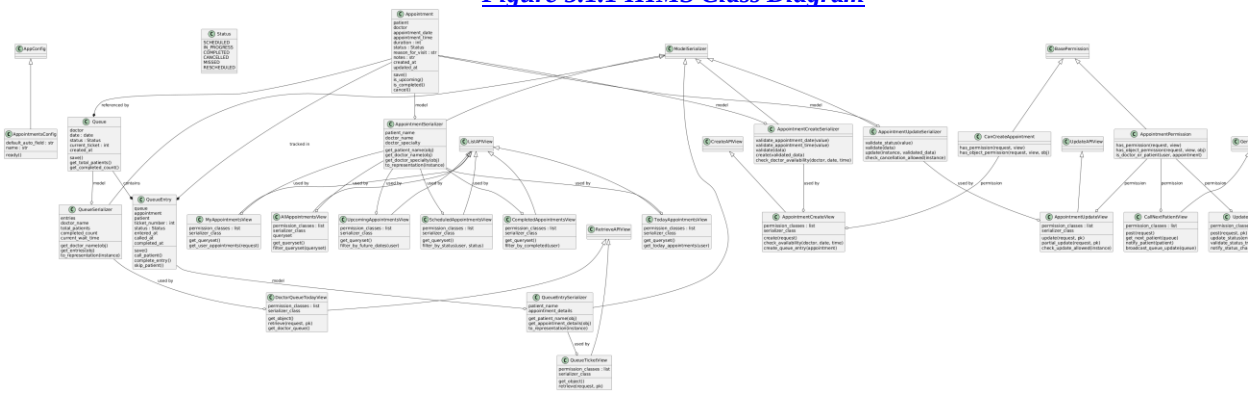

Figure 5.1.2 Appointment Class Diagram

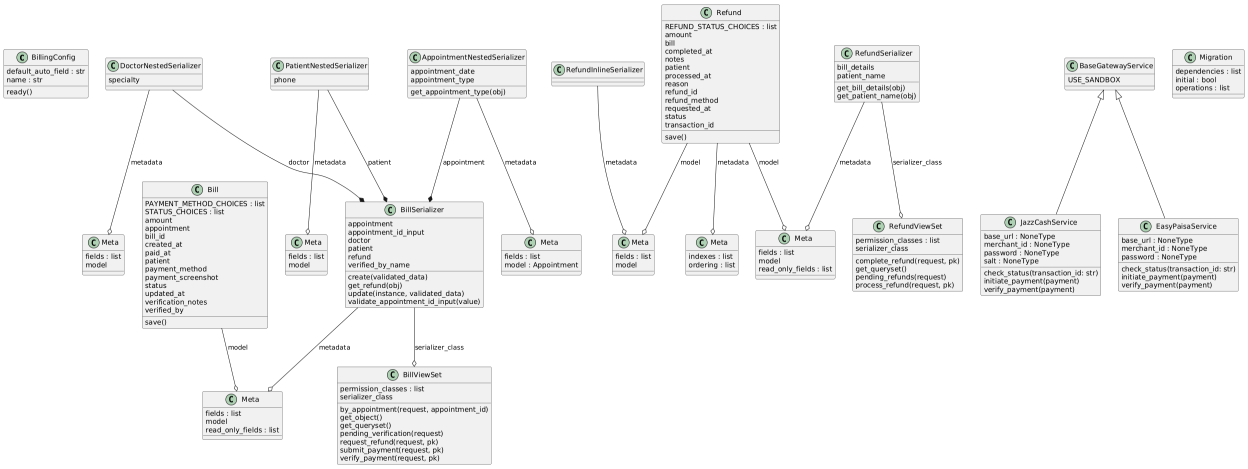

Figure 5.1.3 Billing Class Diagram

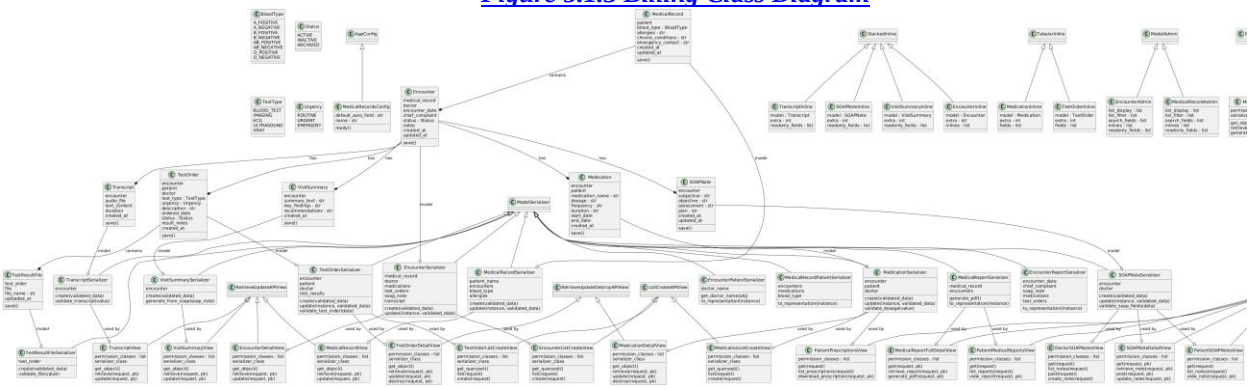

Figure 5.1.4 Medical Records Class Diagram

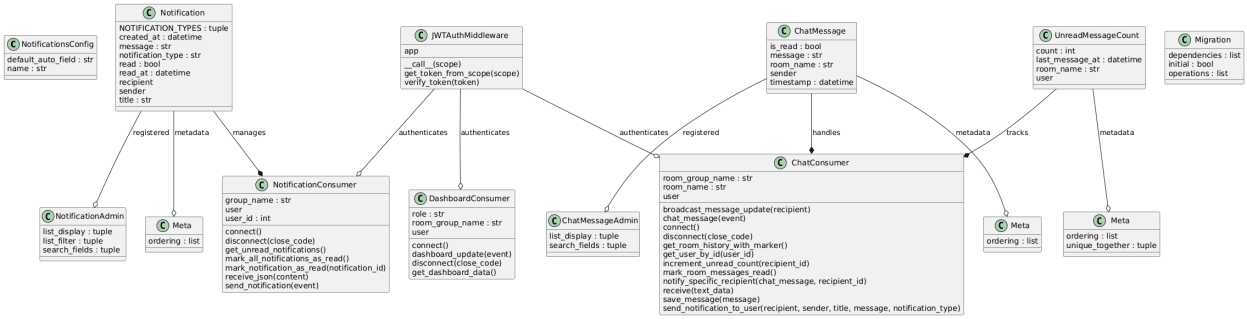

Figure 5.1.5 Notification Class Diagram

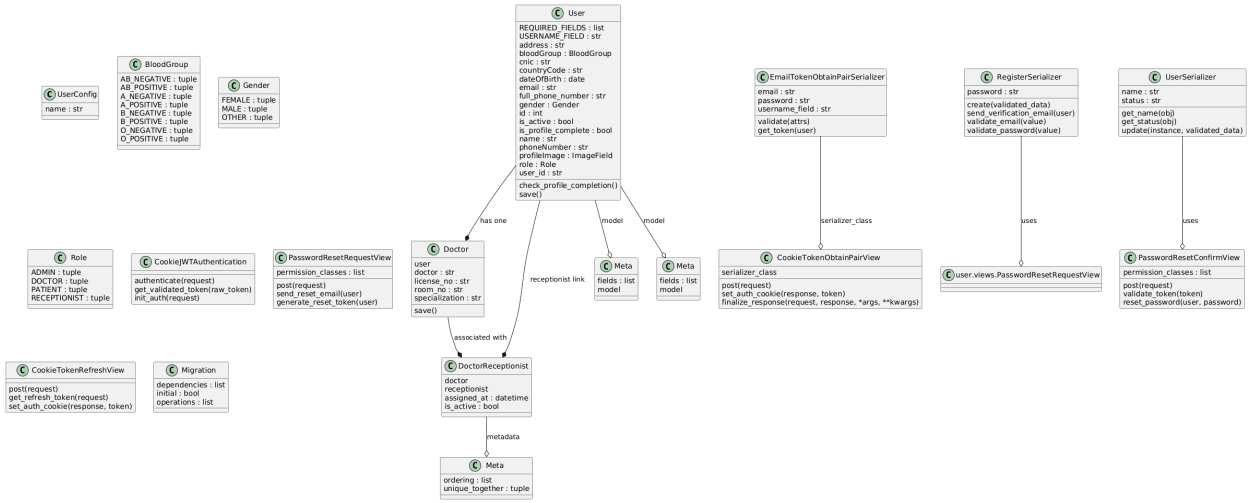

Figure 5.1.6 User Class Diagram

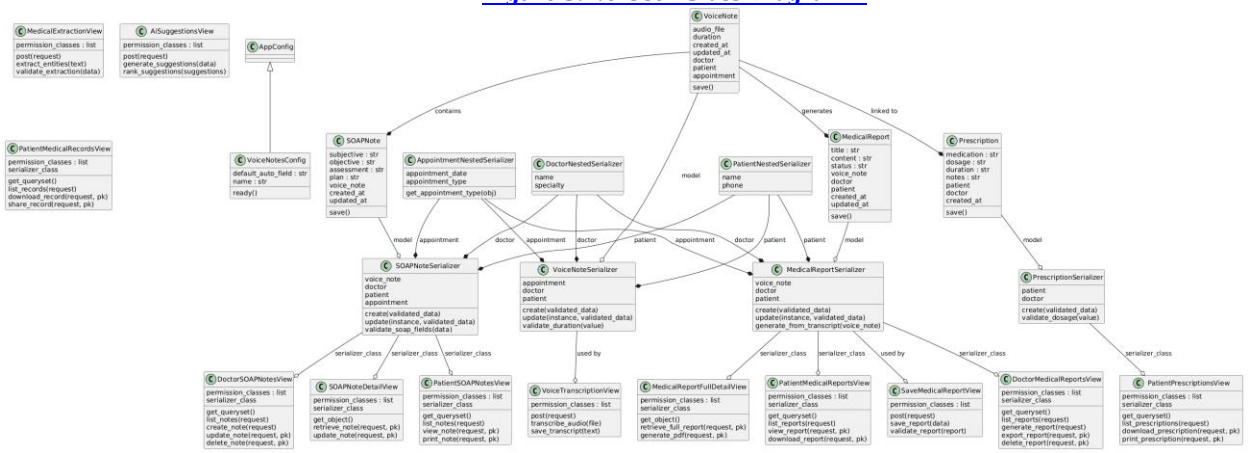

Figure 5.1.7 Voice Notes Class Diagram

5.2 ER Diagram

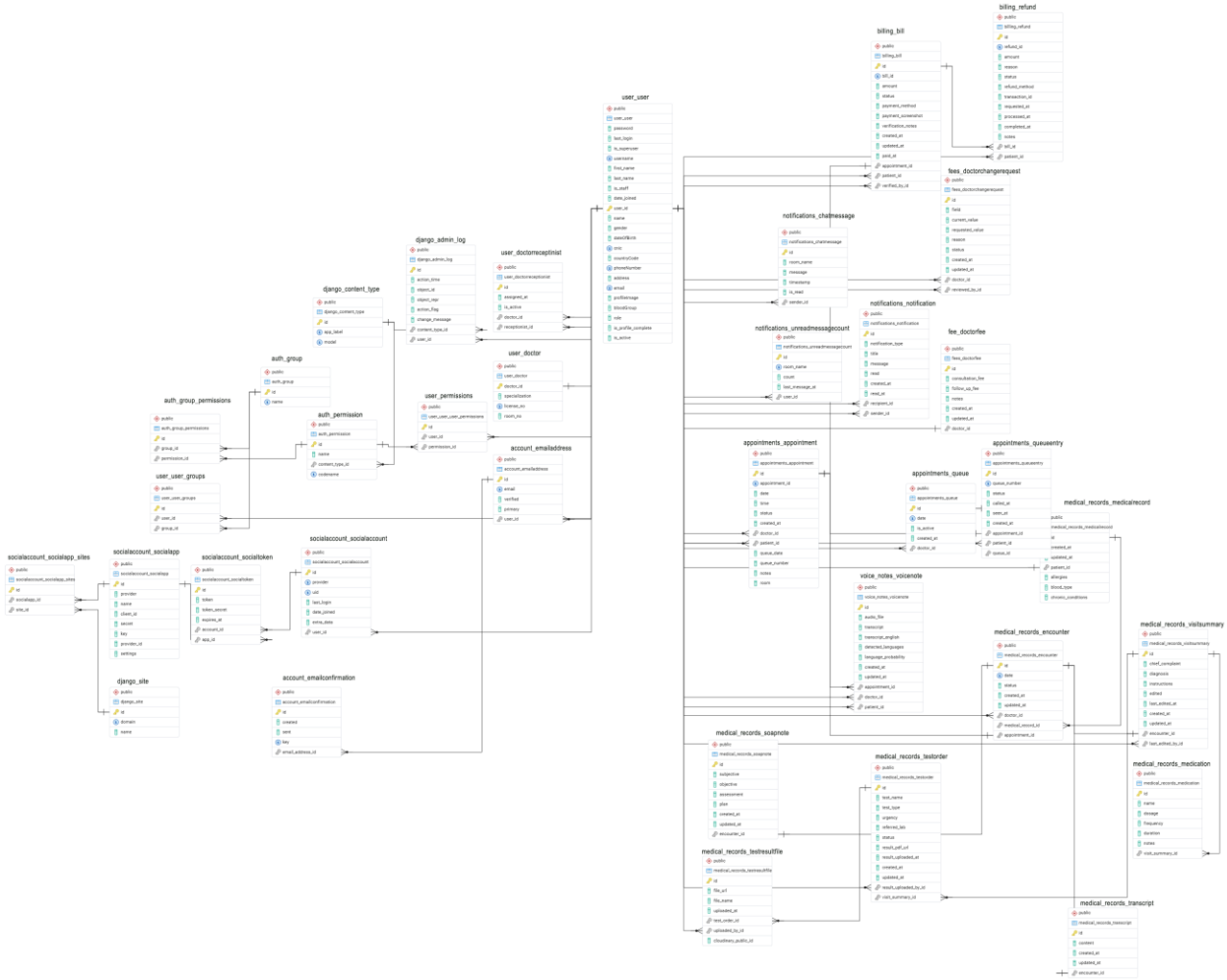

Figure 5.2 ER Diagram

5.3 State Transition Diagram

Figure 5.3 State Transition Diagram

5.4 Sequence Diagram

5.4.1 Sequence Diagram (Login)

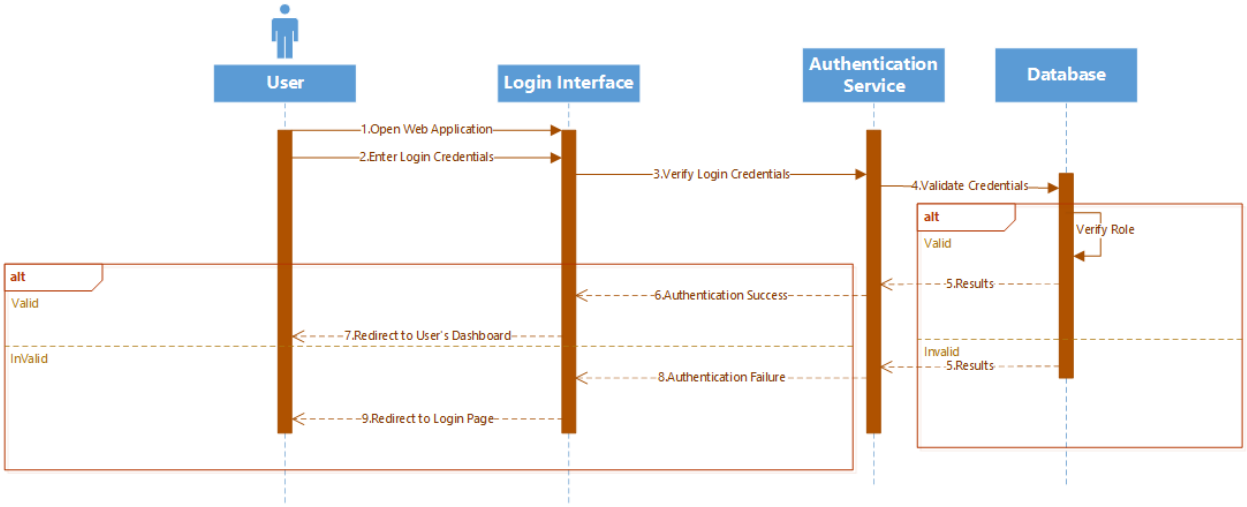

Figure 5.4.1 Sequence Diagram (Login)

5.4.2 Sequence Diagram (Patient)

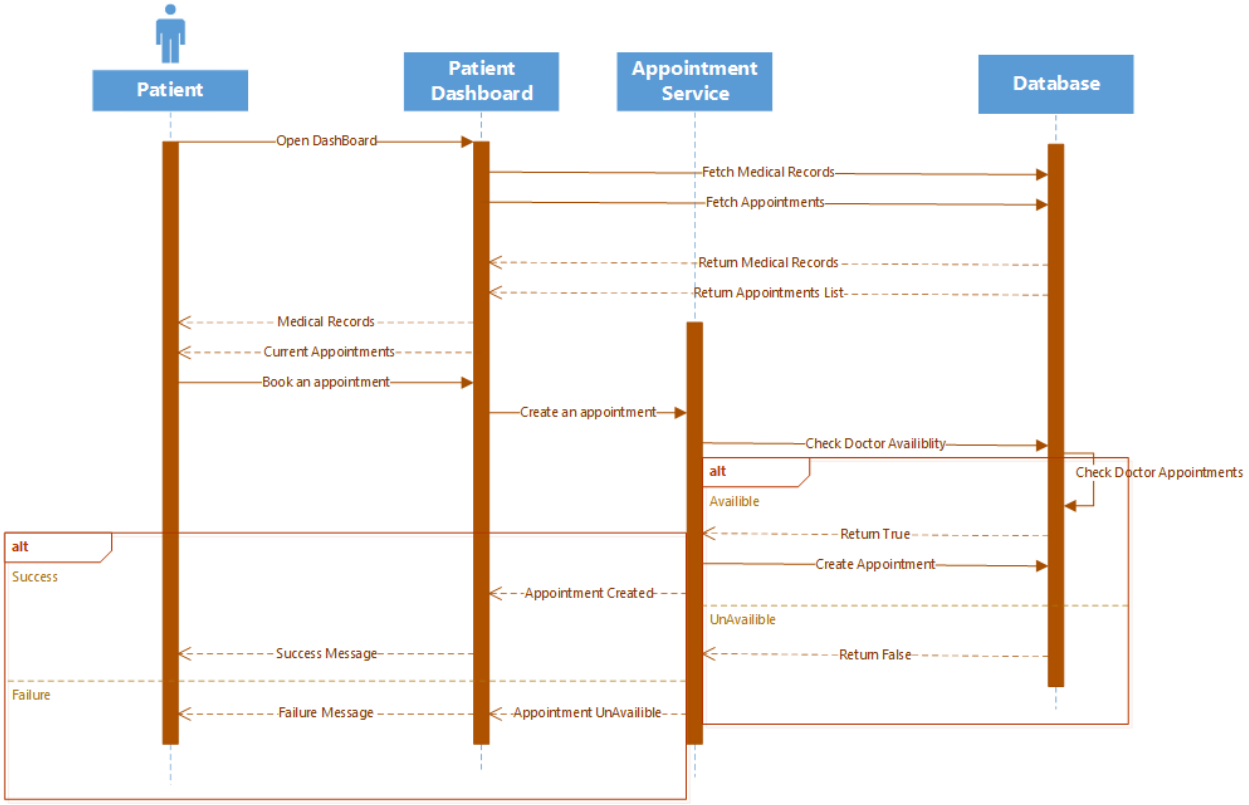

Figure 5.4.2 Sequence Diagram (Patient)

5.4.3 Sequence Diagram (Doctor)

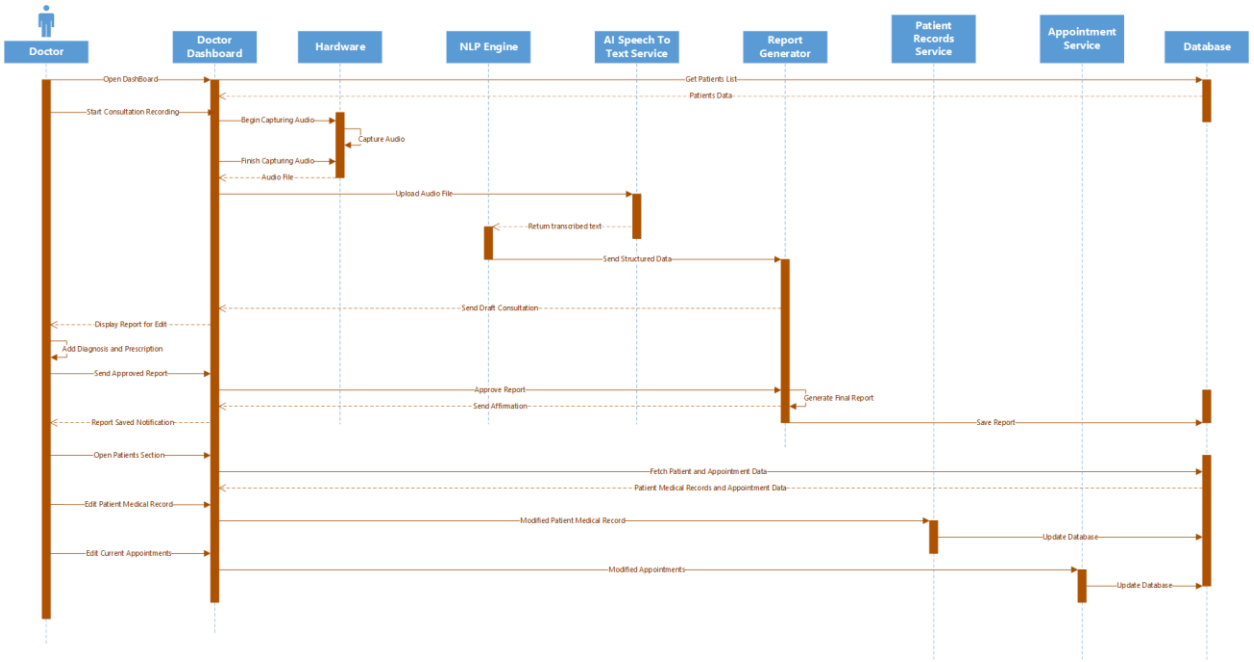

Figure 5.4.3 Sequence Diagram (Doctor)

5.4.4 Sequence Diagram (Receptionist)

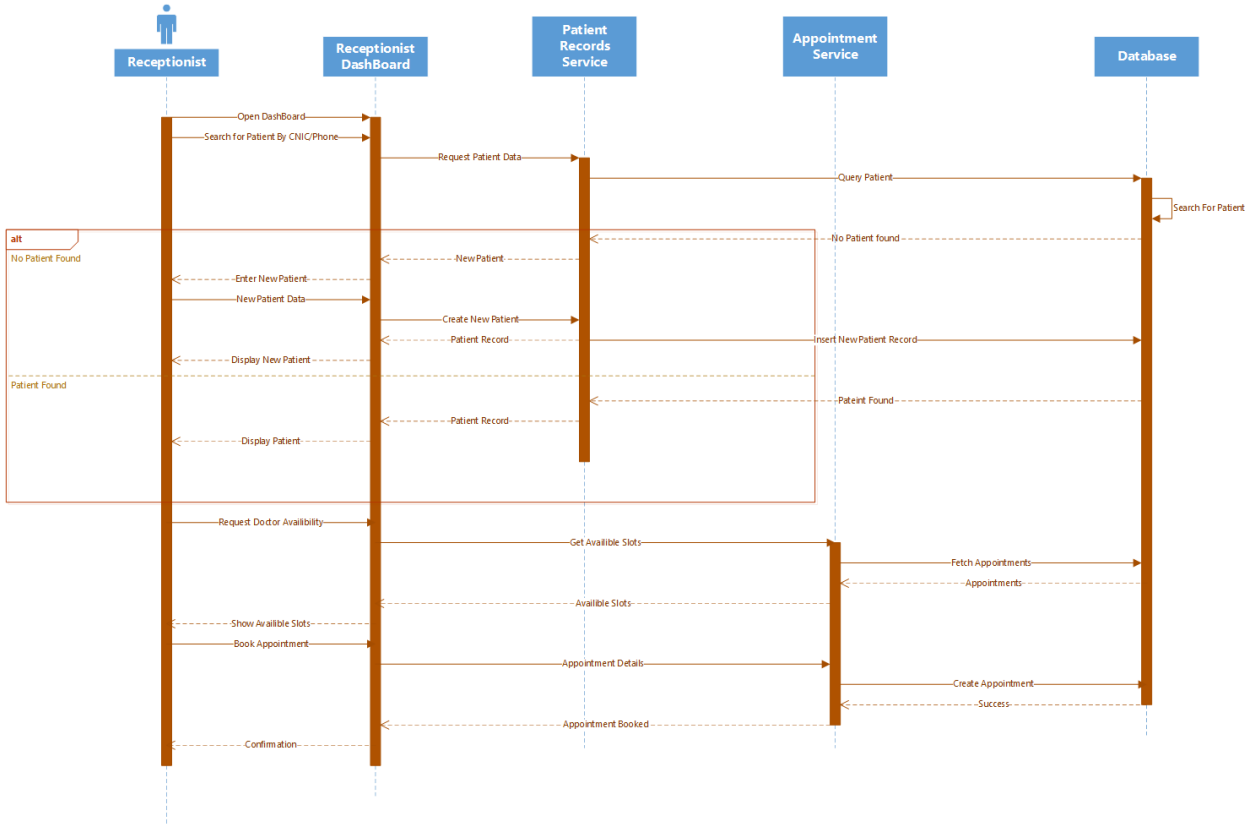

Figure 5.4.4 Sequence Diagram (Receptionist)

5.4.5 Sequence Diagram (Admin)

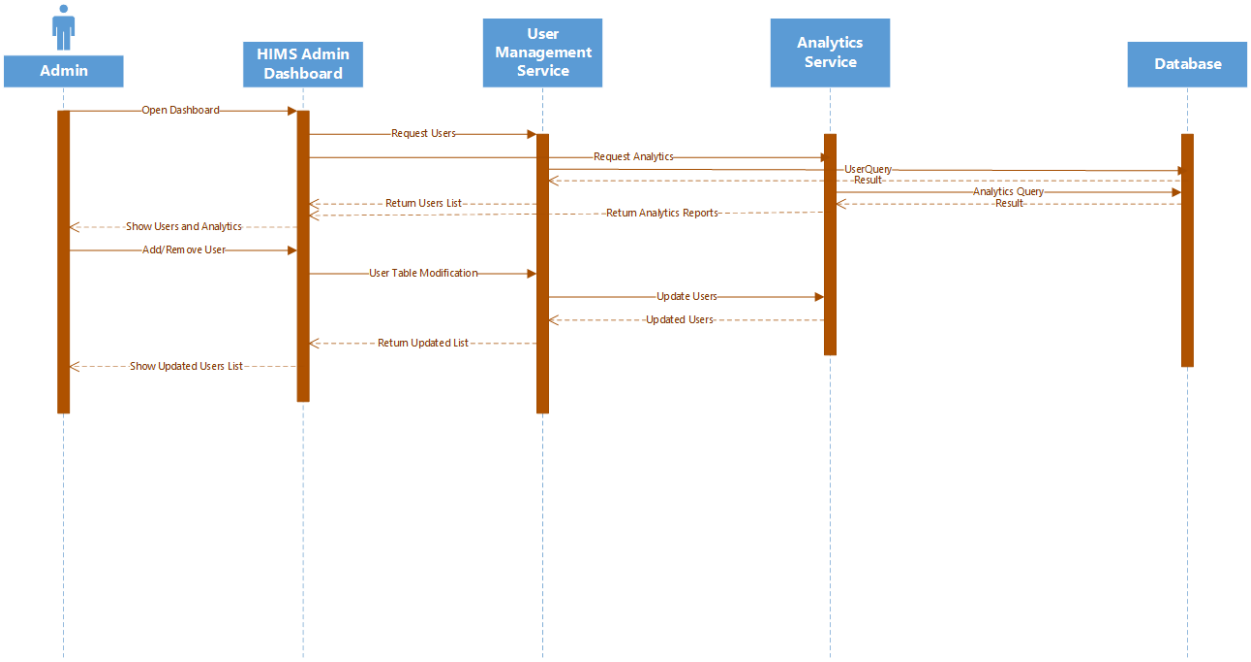

Figure 5.4.5 Sequence Diagram (Admin)

5.5 Data Flow Diagram

5.5.1 DFD Level 0

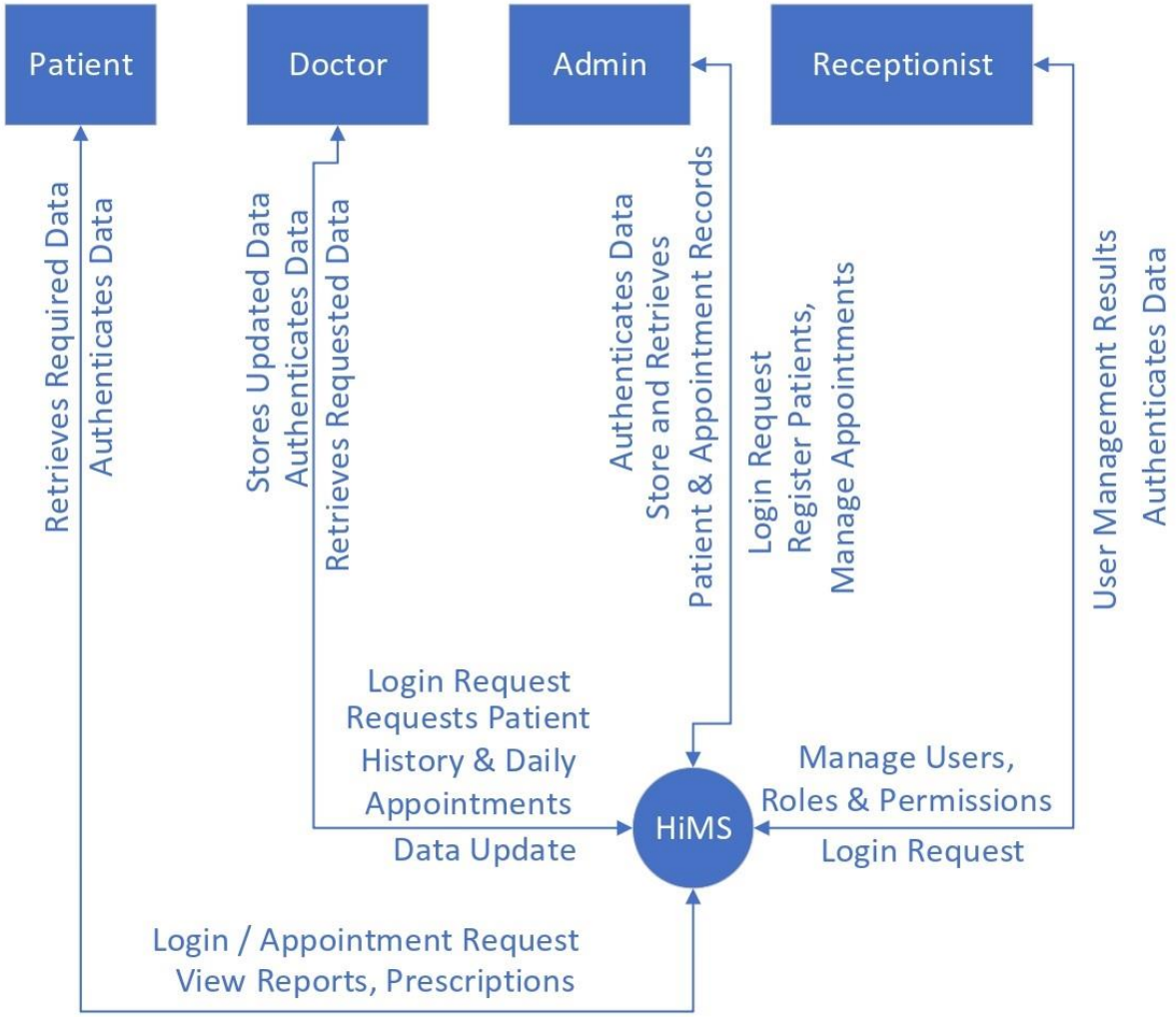

Figure 5.5.1 DFD Diagram (Level 0)

5.5.2 DFD Level 1

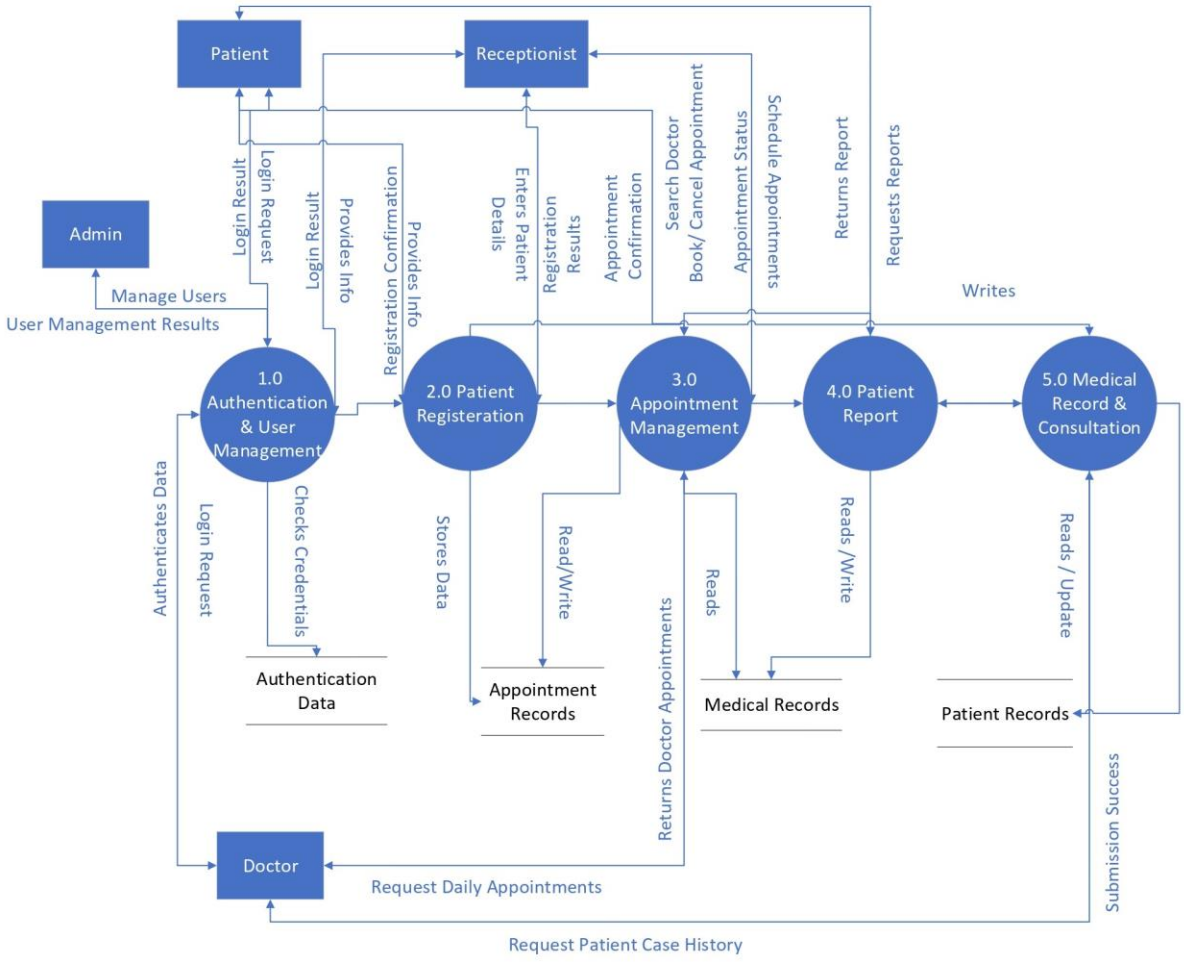

Figure 5.5.2 DFD Diagram (Level 1)

5.5.3 DFD Level 2

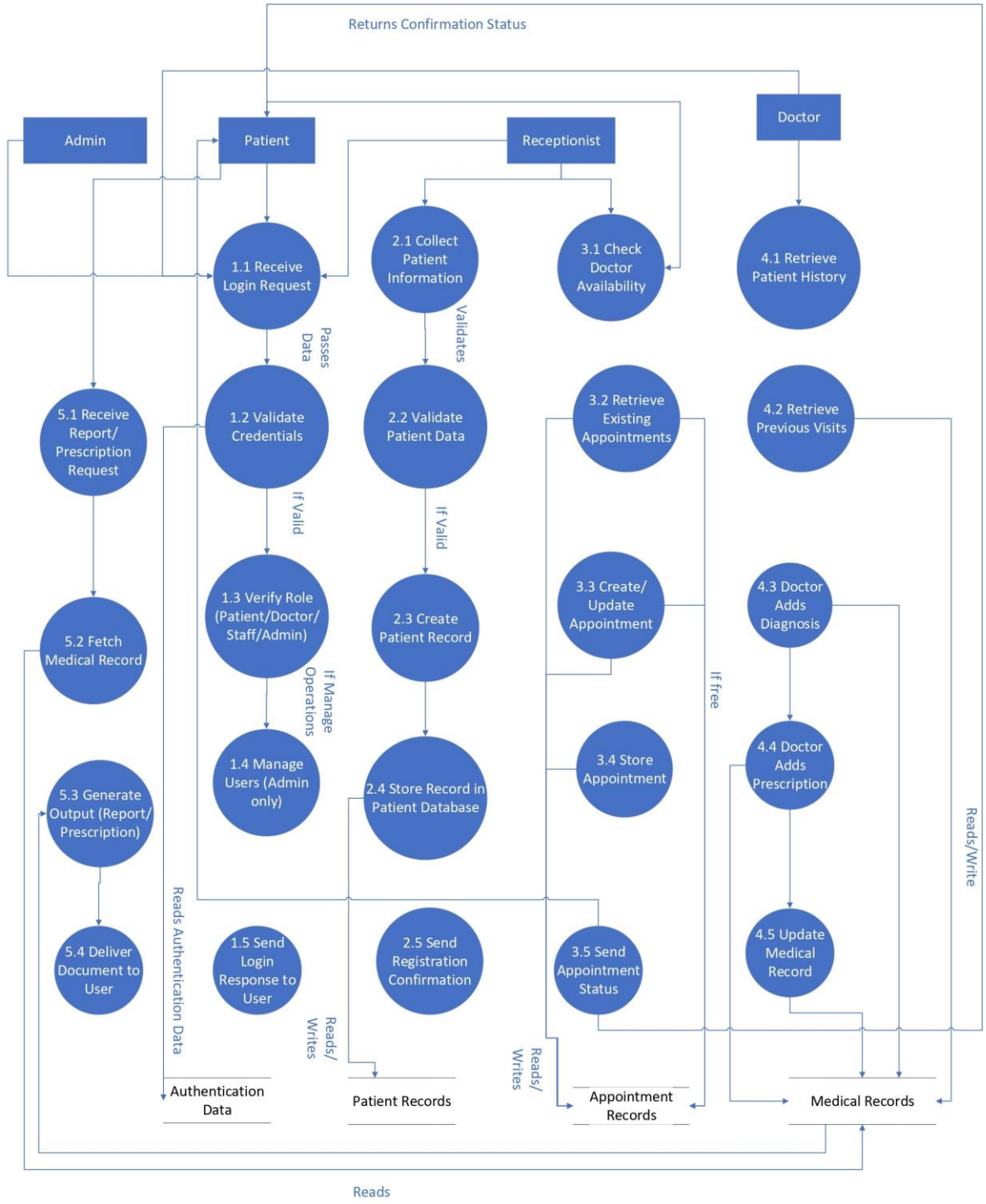

Figure 5.5.3 DFD Diagram (Level 2)

5.6 GUI

5.6.1. Landing Page

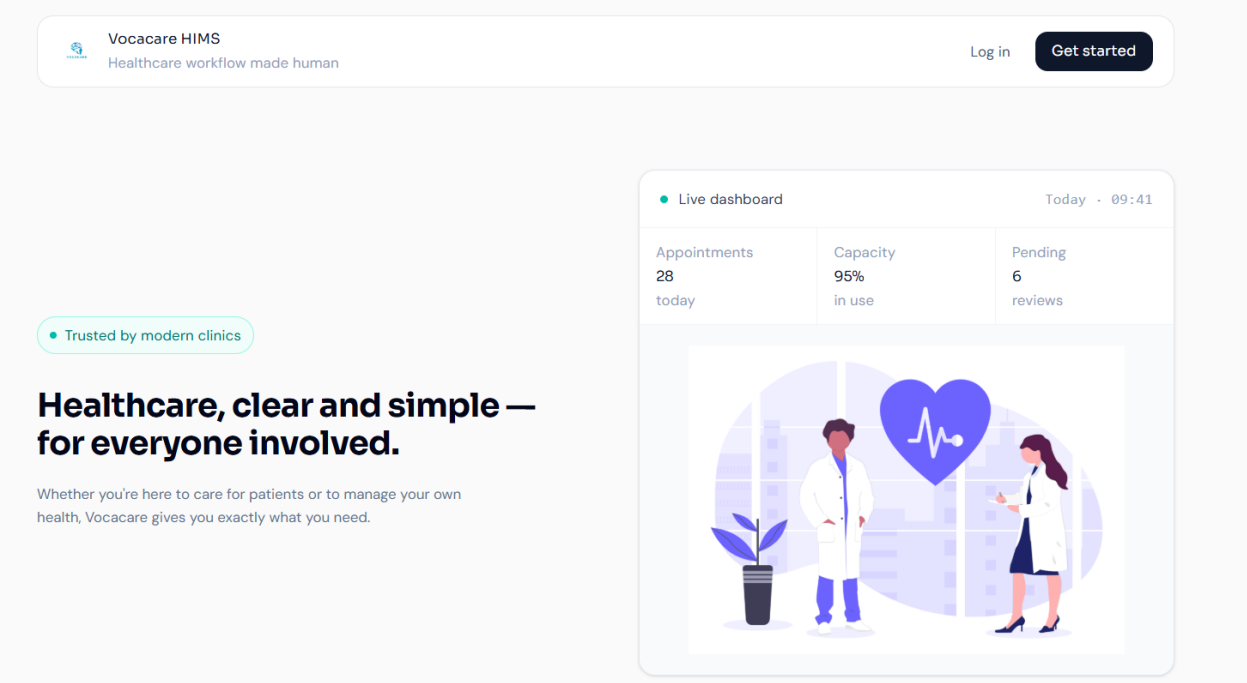

Figure 5.6.1 Landing Page

Purpose & Context:

The Landing Page serves as the primary entry point and first visual impression of the Vocacare system for all user types — doctors, patients, receptionists, and administrators. The page directs users to either register a new account or log into an existing one. No authentication is required to view the landing page, making it fully public facing.

Element Details:

1. Branding Area

- Vocacare logo displayed prominently in the header
- Tagline: "Healthcare, clear and simple – for everyone involved."

2. Primary Actions

- **Sign In Button** → Navigates to the Registration Page where new users can create a Vocacare account by providing their name, email, and password. Patient accounts require admin approval before activation.
- **Log In Button** → Navigates to the Login Page where existing users authenticate using their registered email and password. Upon successful authentication, users are redirected to their role-specific dashboard.

User Flow (Landing Page)

1. **User clicks "Sign In"**

   System Action: Performs a smooth client-side navigation to the Registration Page. The registration form renders with fields for full name, email address, and password. A notice informs the user that patient accounts require administrative approval before activation.

2. **User clicks "Log In"**

   System Action: Performs a smooth client-side navigation to the Login Page. The login form renders with email and password fields, along with a "Log in with Google" option. Upon successful authentication, the backend validates the user's role and redirects to the corresponding dashboard.

3. **User returns to landing**

   System Action: No session or authentication state is retained on the landing page. Navigating back to the root URL results in a fresh render of the landing page with default state. If the user is already authenticated, the system automatically redirects them to their role-specific dashboard instead of showing the landing page again.

5.6.2. Register Page

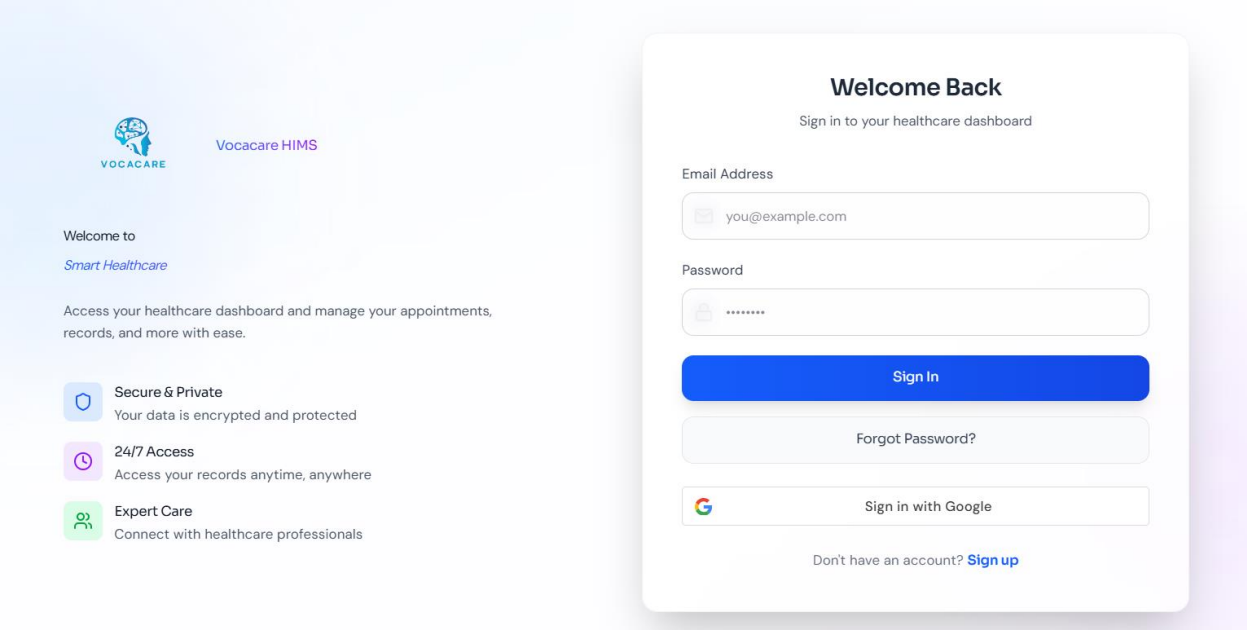

Figure 5.6.2 Register Page

Purpose & Context:

The Registration Page allows new users to create an account. It is accessible to patients who self-register, as well as to staff accounts created through the admin panel. The form collects essential identity and credential information. Patient accounts submitted through this page remain inactive

until reviewed and approved by an authorized administrator, ensuring only verified individuals gain system access.

Element Details:

Register Form

- Full Name input field (required)
- Email Address input field with format validation (required)
- Password input field with visibility toggle (minimum 8 characters)
- Confirm Password field to prevent entry errors
- Register Button — submits the form and triggers the account creation API
- Link to Login Page for users who already have an account

User Flow

1. User fills in name, email, and password → Clicks Register System Actions:

- Validates that all required fields are filled
- Validates email format using regex
- Checks that password meets minimum length requirements and matches the confirm password field
- Sends credentials to the registration API endpoint
- If successful → displays a confirmation message: "Account created. Awaiting admin approval." (for patient role)
- If email already exists → shows inline error: "An account with this email already exists."
- If passwords do not match → shows inline error: "Passwords do not match."

5.6.3. Login Page

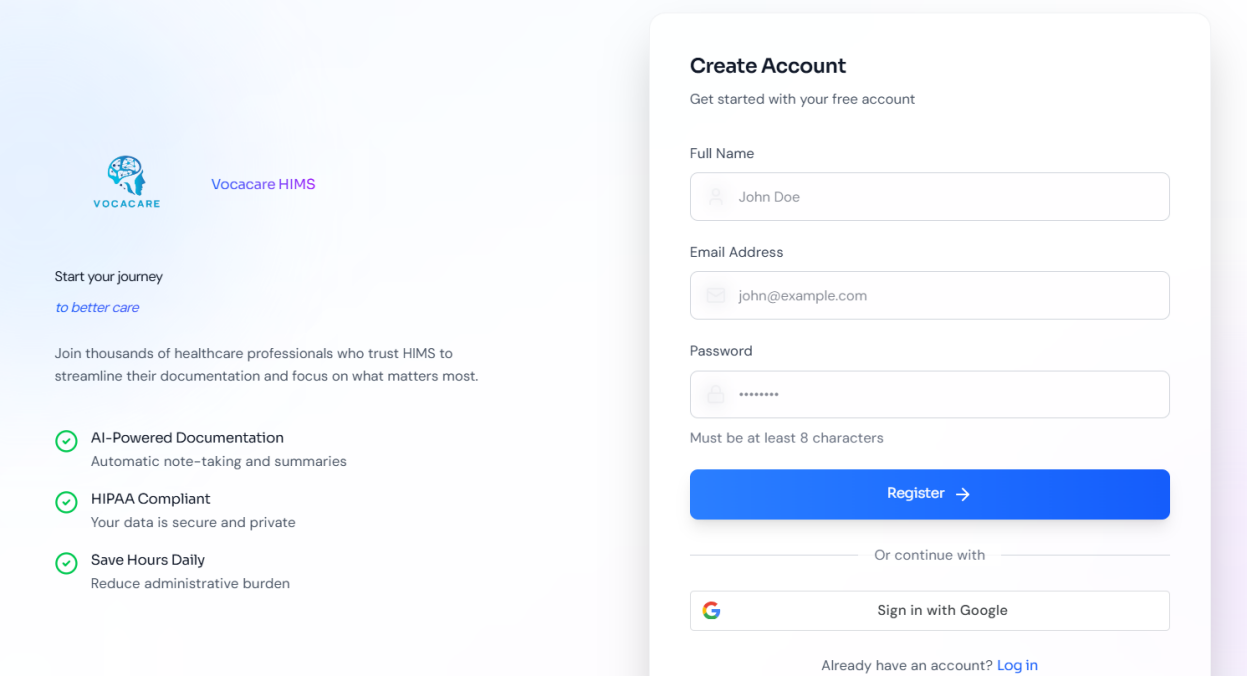

Figure 5.6.3 Login Page

Purpose & Context:

The Login Page provides secure, role-aware authentication for all system users. It supports both standard email/password login and Google OAuth sign-in. Upon successful authentication, the backend identifies the user's assigned role and redirects them directly to their corresponding dashboard — eliminating manual navigation. Failed login attempts display clear, inline error messages to guide the user.

Element Details:

Login Form

- Email input field with format validation
- Password input field with show/hide toggle
- Sign In button — submits credentials to the authentication API
- "Log in with Google" button — initiates OAuth 2.0 Google sign-in flow
- Inline error message area for invalid credentials
- "Forgot Password?" link for password recovery flow
- Link to Register Page for new users

User Flow

1. Enter Email + Password → Click Sign In System Actions:

- Validates email format on the frontend before submission
- Validates that password field is not empty
- Sends credentials to the authentication API
- If valid → backend returns a session token; frontend stores it in an HTTP-only cookie and redirects the user to their role-specific dashboard (Doctor, Patient, Receptionist, or Admin)
- If invalid → displays inline error: "Incorrect email or password."
- If account is pending approval → displays: "Your account is awaiting admin approval."

2. Enter Name + Email + Password → Click Register System Actions:

- Validates all fields client-side before submission
- Sends registration data to the API
- If valid → creates account and shows confirmation message
- If invalid → shows inline error: "Invalid email or password format."

5.6.3 Patient Dashboard

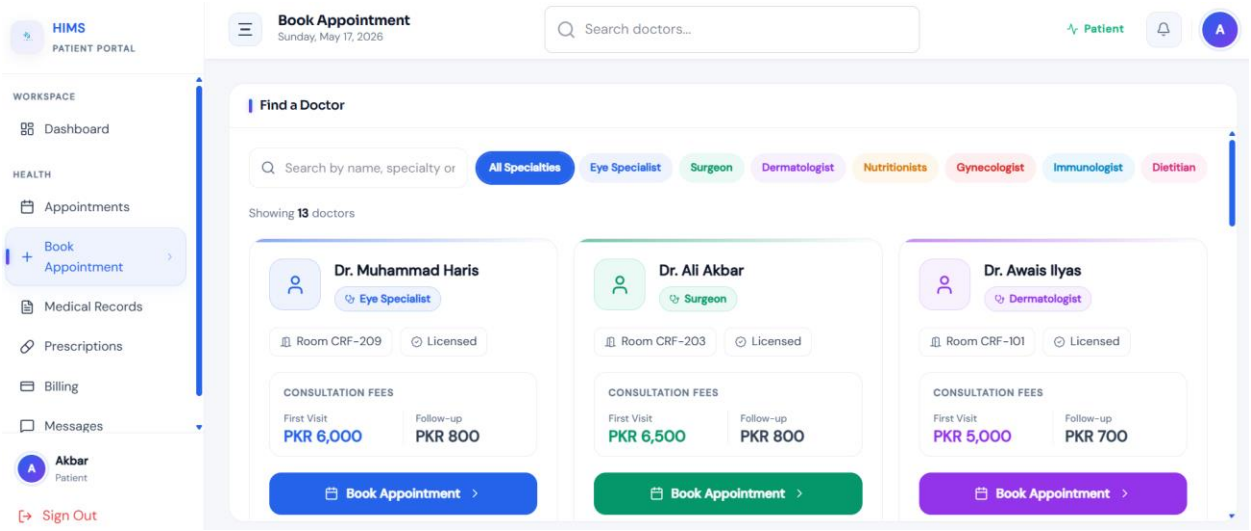

Figure 5.6.4 Patient Dashboard

Purpose & Context:

The Patient Dashboard is the central interface for patients to manage their healthcare interactions with the hospital. It provides a unified, easy-to-navigate view of appointments, medical history, prescriptions, and billing information. The dashboard is designed to be accessible to users with varying levels of technical familiarity, offering clear action buttons and organized module cards.

All data displayed is personal to the authenticated patient and retrieved securely from the backend.

Element Details:

Header

- Vocacare logo and platform name
- Personalized welcome message (e.g., "Welcome, [Patient Name]")
- Global search bar for finding doctors or appointment records
- Notification bell with dropdown for unread alerts
- User profile avatar with dropdown menu (Profile, Settings, Logout)

Modules

- **Upcoming Appointments** — displays scheduled appointments with doctor name, date, time, and queue position; allows cancellation
- **Medical History** — chronological timeline of past consultations with expandable detail views for diagnoses
- **Prescriptions** — list of active and past prescriptions with download option as PDF
- **Billing Summary** — overview of pending and paid invoices with payment action buttons
- **Quick Actions** — shortcut buttons for Book Appointment, View Reports, Contact Reception

User Flow

1. **Patient books an appointment**:

   System Action: Opens the appointment booking panel with a calendar view. Patient selects a doctor and available date slot. System checks for conflicts and confirms the slot. Appointment appears in the Upcoming Appointments module with an assigned queue number.

2. **Patient views medical history:**

   System Action: Loads the patient's personal medical timeline from the database. Each entry displays date, attending doctor, and diagnosis summary. Clicking an entry expands the full consultation detail.

3. **Patient pays invoice**:

   System Action: Redirects to the Billing section. Patient selects the outstanding invoice, chooses a payment method (cash, JazzCash, or Easypaisa), and confirms. Payment status is updated in real-time and a receipt is generated.

4. **Patient clicks notification bell**:

   System Action: Dropdown opens showing unread notifications (appointment reminders, billing alerts, doctor messages). All notifications in the dropdown are marked as read upon viewing.

5. **Patient edits profile**:

   System Action: Opens the profile edit form with pre-filled current information. Patient updates desired fields and clicks Save. The interface refreshes to reflect the updated profile data.

5.6.4 Receptionist Dashboard

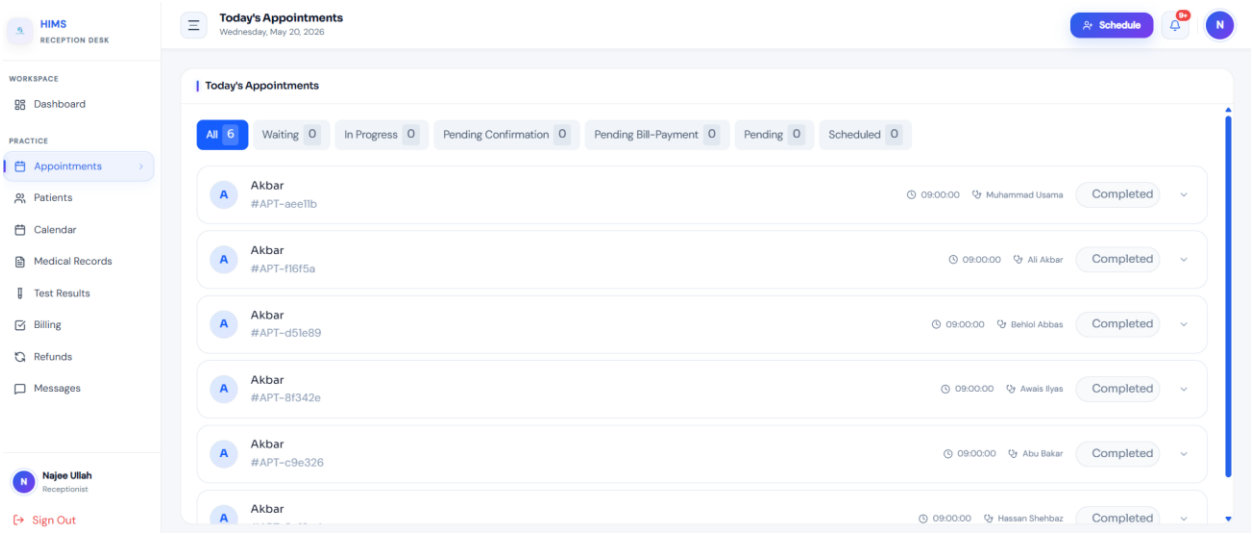

Figure 5.6.5 Receptionist Dashboard

Purpose & Context:

The Receptionist Dashboard is designed to support front-desk operations efficiently. Each receptionist is assigned to a specific doctor and can only view and manage operations related to that doctor's patient queue and appointments. The interface is kept simple and task-focused, enabling receptionists to handle patient check-ins, appointment bookings, registrations, and schedule monitoring without requiring deep technical knowledge.

Element Details:

Header

- Vocacare logo and welcome message
- Patient search bar for quick lookup
- Current time and date display

- Notification bell with dropdown
- User menu (Profile, Settings, Logout)

Main Modules

- **Today's Appointments** — table view of all scheduled patients for the assigned doctor, showing queue position, patient name, appointment time, and status
- **Patient Search** — search box with live results for looking up registered patients
- **Quick Actions** — shortcut buttons for Register New Patient and Book Appointment
- **Appointment Overview Table** — filterable list with check-in, reschedule, and cancel actions

User Flow

1. **Receptionist searches for a patient**:

   System Action: Queries the patient database in real-time as the receptionist types. Displays a list of matching results with patient name and ID. Clicking a result opens the full patient profile for the receptionist's review and action.

2. **Receptionist clicks "Check-In":**

   System Action: Marks the selected appointment as "Arrived" in the system. Updates the doctor's live appointment queue to reflect the patient's arrival.

3. **Click "Book Appointment":**

   System Action: Opens the appointment booking form. If a patient was previously selected, their information is pre-filled. The receptionist selects an available date slot from the doctor's schedule and confirms the booking.

4. **Clicking Notifications Icon:**

   System Action: Dropdown opens displaying system alerts such as new appointment requests or queue updates. All viewed notifications are marked as read automatically.

5. **Clicking User Profile**:

   System Action: Opens a dropdown menu with options for Profile settings and Logout. Selecting Logout clears the active session and redirects to the Login page.

5.6.5 Doctor Dashboard

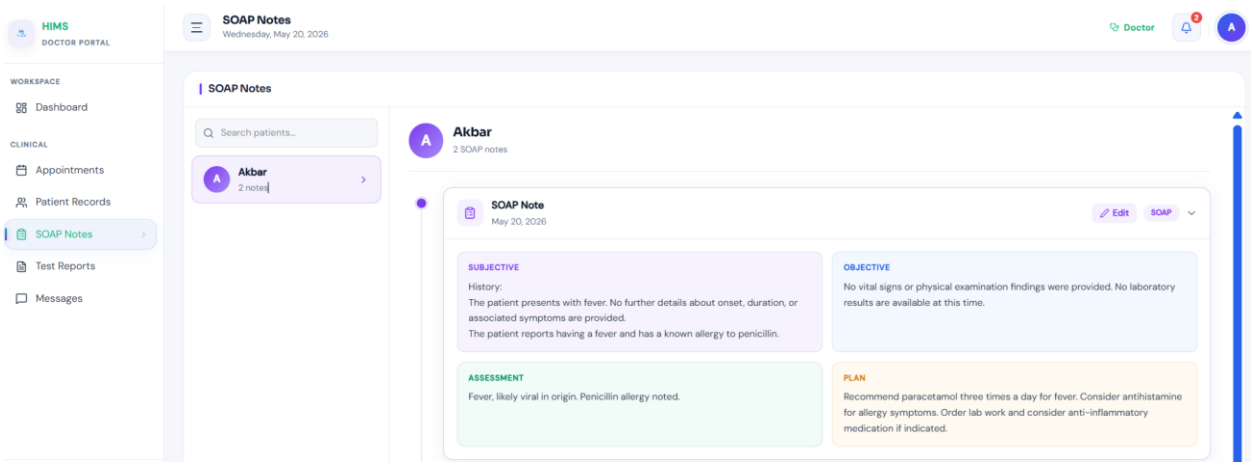

Figure 5.6.6 Doctor Dashboard

Purpose & Context:

The Doctor Dashboard is the primary workspace for doctors to manage their daily consultations, patient records, and AI-assisted documentation. It is designed to minimize administrative friction and keep the doctor focused on patient care. The dashboard provides a real-time view of the day's appointment queue, quick access to patient histories, and an integrated SOAP note editor with voice recording capabilities powered by the Vocacare AI engine.

Element Details:

Header

- Vocacare logo and welcome message with doctor's name
- Current time and date display
- Notification bell with alert dropdown
- User menu (Profile, Settings, Logout)

Tabs

- **Today's Appointments** — lists all scheduled patients for the current day with status indicators (Waiting, In Session, Completed, Missed)
- **My Patients** — full registry of doctor's patient list with search and filter capabilities
- **Profile** — doctor's personal and professional information management

Session Modal

- Left panel: Patient history summary (previous diagnoses, medications, visit dates)
- Center panel: SOAP notes editor (Subjective, Objective, Assessment, Plan) with auto-save
- Right panel: Voice recorder controls (Start, Stop, Playback)

- Action buttons: Submit Session, Finish Session

User Flow

1. **Doctor clicks "Start Session"**:

   System Action: Opens the fullscreen session modal. Loads the selected patient's medical history in the left panel and initializes the SOAP note template in the center. Requests microphone permissions if not already granted.

2. **Doctor edits SOAP Notes:**

   System Action: All changes to the SOAP note fields are auto-saved as a local draft. Unsaved or modified fields are visually highlighted to prevent accidental data loss.

3. **Doctor clicks "Start Recording":**

   System Action: Requests microphone access from the browser. Upon permission, begins audio capture and sends the stream to the AI transcription engine. A pulsing "Recording…" indicator is displayed to signal active capture.

4. **Doctor clicks "Stop Recording"**:

   System Action: Finalizes the audio capture and saves the audio blob. The AI engine processes the recording, extracts medical entities (symptoms, diagnosis, medications), and populates the corresponding SOAP note fields for doctor review.

5. **Doctor clicks "Submit":**

   System Action: Validates all required SOAP fields. Uploads the completed notes and audio recording to the backend. Closes the session modal, marks the appointment status as "Completed," and displays a success toast: "Session Submitted."

6. **Doctor clicks "Finish Session"**:

   System Action: Closes the session modal without submitting if called prematurely, returning the doctor to the Appointments tab. If session was already submitted, it simply dismisses the modal.

7. **Doctor searches for a patient:**

   System Action: Triggers a live global search across the doctor's patient registry. Results appear in a dropdown as the doctor types. Clicking a result opens that patient's full profile page.

8. **Editing profile**:

   System Action: Allows the doctor to update personal and professional information (name, specialization, contact). Changes are saved via API and the UI refreshes to reflect the updates.

5.6.6 Admin Dashboard

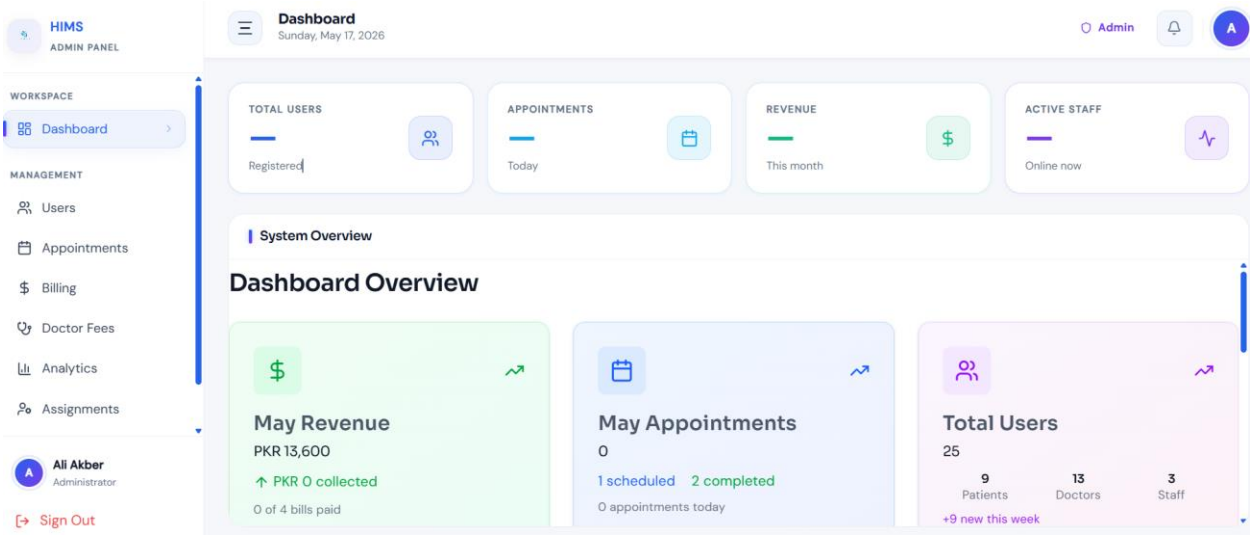

Figure 5.6.7 Admin Dashboard

Purpose & Context:

The Admin Dashboard provides hospital administrators with complete oversight and control of the Vocacare system. It serves as the management hub for user accounts, doctor-receptionist assignments, hospital analytics, billing monitoring, and system activity logs. The dashboard is data-rich and organized into clearly separated sections, enabling administrators to make informed operational decisions and maintain system integrity without requiring direct technical access to the backend.

Element Details:

Header

- Welcome message with admin name and role
- Global search bar in Users Tab(searches across users, departments, doctors, and logs)
- Current time and date display
- Notification bell with alert dropdown
- Admin menu (Profile, Logout)

Main Sections

- **Overview** — summary cards showing total patients, active doctors, today's appointments, and pending account approvals
- **Users** — full user management table with options to add, edit, activate, deactivate, or delete accounts; also handles patient account approval

- **Appointments** — system-wide appointment monitoring across all doctors with filter and export options
- **Billing** — overview of revenue, outstanding invoices, and payment history with export to PDF/CSV
- **Analytics** — charts and graphs for patient inflow trends, doctor workload, appointment frequency, and operational performance metrics

User Flow

1. **Admin uses global search:**

   System Action: Searches across all entities in the system via users tab — users, doctors, receptionists.

2. **Admin approves patient account**:

   System Action: Pending patient accounts appear in the Users section with a toggle action button which activates the account of the user.

3. **Admin views analytics:**

   System Action: Loads interactive charts showing appointment trends, revenue data.

4. **Admin edits system roles:**

   System Action: Opens the role and permissions editor. Admin can assign or revoke access to specific modules for each role type. Changes are saved and applied immediately across the system.

5. **Notifications dropdown:**

   System Action: Displays system-wide alerts including new account approval requests, failed system events, and billing anomalies. Events are marked as read upon being viewed.

6. **Logout**:

   System Action: Clears the admin session token from the cookie store and redirects to the Login page. All protected routes become inaccessible until re-authentication.

2. References

Ref. No.	Document Title	Date of Release/ Publication	Document Source
PGBH01-2025-Proposal	Project Proposal	Sep 23, 2025	https://github.com/Hospital-Information-Management-System/Capstone/tree/main/Project%20Proposal

Ref. No.	Document Title	Date of Release/ Publication	Document Source
PGBH01-2025-FS	Functional Specification	Oct 11, 2025	https://github.com/Hospital-Information-Management-System/Capstone/tree/main/Project%20SRS
PGBH01-2025-FS	Design Document	Dec 6, 2025	https://github.com/Hospital-Information-Management-System/Capstone/tree/main/Project%20Design%20Document